

Documentación técnica del notebook

Práctica B4-T2 — XAI: construcción, optimización y auditoría de un modelo de concesión de crédito

Máster MIAX — Módulo B4, Tema 2

Julio de 2026

Contents

Cómo leer este documento	2
Parte 0 — Portada y resumen ejecutivo del notebook	3
Parte 1 — Introducción, convención de costes y configuración del entorno	4
1.1 Por qué se fija una convención de costes explícita	4
1.2 Instalación de dependencias	4
1.3 Imports, semilla y generador aleatorio	4
Parte 2 — Carga y exploración de datos (EDA)	6
2.1 Carga de los tres ficheros	6
2.2 Inventario de tipos, nulos y rango de valores	6
2.3 Desbalanceo del target y coste de las estrategias triviales	7
2.4 Distribuciones y detección de códigos centinela	7
2.5 Missingness informativo	8
2.6 Utilización de crédito: señal fuerte y probable error de unidad	8
2.7 Duplicados exactos y edades imposibles	9
2.8 Matriz de correlaciones y el artefacto de los códigos centinela	9
2.9 Ausencia de drift entre construcción y producción	10
Parte 3 — Preprocesamiento	11
3.1 La función prepara_features: la única puerta de entrada de datos	11
3.2 Limpieza de filas: solo en construcción, nunca en producción	11
3.3 Esquema de validación: por qué 80/20 más 5-fold OOF	12
3.4 El pipeline de la regresión logística: winsorización, imputación y escalado	12
3.5 Interpretación de la sección 3	14
Parte 4 — Modelado, escenario 1 (C_FP = C_FN = 1)	15
4.1 La maquinaria de coste: coste_medio, curva_coste, umbral_optimo	15
4.2 Comparativa de candidatos por validación cruzada	16
4.3 Elección del modelo ganador y curva de coste del escenario 1	18
4.4 Evaluación única en test	18
Parte 5 — Modelado, escenario 2 (C_FP = 1, C_FN = 10)	20
5.1 La comparativa de siete estrategias cost-sensitive	20
5.2 Diagrama de fiabilidad (calibración)	21
5.3 Evaluación única en test del escenario 2	21

Parte 6 — Comparación de escenarios y sensibilidad a la matriz de coste	23
6.1 Tabla resumen y curvas de coste lado a lado	23
6.2 Análisis de sensibilidad al ratio de coste	23
Parte 7 — Auditoría (I): modelo subrogado con reglas	25
7.1 El extractor de reglas: recorrido recursivo del árbol interno de sklearn	25
7.2 Búsqueda de la profundidad óptima del subrogado	26
7.3 El árbol final y sus reglas	26
7.4 La trampa de la fidelidad con clases desbalanceadas	27
Parte 8 — Auditoría (II): contrafactuales	28
8.1 Selección determinista de los seis clientes	28
8.2 El buscador exhaustivo de contrafactuales propio	29
8.3 Aplicación de la búsqueda propia a los clientes denegados	31
8.4 Contraste con DiCE y el descubrimiento del reseteo silencioso del umbral	31
8.5 Contrafactuales inversos para los clientes aprobados	33
8.6 Las cartas al cliente	33
Parte 9 — Auditoría (III): SHAP global y local	34
9.1 SHAP global: TreeExplainer y el gráfico de enjambre (<i>beeswarm</i>)	34
9.2 Dependence plots de las variables más influyentes	34
9.3 Explicación local de los seis clientes de la sección 8	35
9.4 El cruce cuantitativo entre contrafactuales y SHAP	35
9.5 Análisis de equidad por tramo de edad	36
9.6 Estabilidad de las explicaciones entre escenarios	37
Parte 10 — Discusión: el enfoque de multi-armed bandit	39
Parte 11 — Generación de los ficheros de producción	41
11.1 Umbrales finales y modelo final	41
11.2 Escritura de los CSV y verificación de integridad	41
Parte 12 — Reflexión final crítica	43
Cierre: reproducibilidad y cómo re-ejecutar el notebook	44

Cómo leer este documento

Este documento recorre, celda a celda, el notebook `practica_B4T2_XAI.ipynb` que resuelve la práctica de credit scoring con auditoría XAI. El objetivo no es repetir el notebook, sino **explicar en detalle qué hace cada bloque de código y, sobre todo, por qué se ha escrito así**: qué alternativa se descartó, qué hallazgo de los datos motivó la decisión, y qué principio de teoría estadística, de aprendizaje automático o de gobernanza de modelos la respalda.

La estructura sigue exactamente las doce secciones del notebook. Cada apartado reproduce el código relevante en bloques python y lo diseca inmediatamente después. Cuando una celda define una función auxiliar (“helper”), se explica su algoritmo con el mismo nivel de detalle que se esperaría en una revisión de código de producción: complejidad, casos límite y por qué no se usó una alternativa más simple (o más complicada).

Para seguir el documento junto al notebook, cada sección indica entre corchetes el número de celda al que corresponde, por ejemplo `[celda 33]`.

Parte 0 — Portada y resumen ejecutivo del notebook

Las dos primeras celdas del notebook ([celda 0] y [celda 1], ambas markdown) no contienen código ejecutable: son la portada con los nombres del grupo y una **tabla de resumen ejecutivo** que se rellena a mano con los números finales que producen las secciones 4, 5 y 11. Colocar este resumen al principio no es un capricho estético — responde directamente a que el 50% de la nota depende del “coste promedio del modelo en el dataset de producción”, así que un corrector debe poder encontrar esa cifra en los primeros 30 segundos de lectura, sin tener que ejecutar nada ni buscar entre 89 celdas. La tabla se completa citando literalmente los valores que calculan las celdas de código correspondientes (nunca escritos “a mano” en otro sitio del notebook), de modo que portada y cuerpo del documento no puedan divergir por un descuido de edición.

Parte 1 — Introducción, convención de costes y configuración del entorno

1.1 Por qué se fija una convención de costes explícita

Antes de escribir una sola línea de código de modelado, el notebook dedica una celda markdown completa ([celda 2]) a fijar sin ambigüedad qué significa “falso positivo” y “falso negativo” en este problema. Esto no es relleno: el notebook de partida que proporciona el profesor resuelve un problema de tumores donde la clase 0 es la maligna, así que su código define `PENALIZACION_FALSO_POSITIVO` y `PENALIZACION_FALSO_NEGATIVO` con la convención invertida respecto a la que es natural para crédito. Adaptar ese código sin releer su lógica de signos llevaría, con altísima probabilidad, a construir un modelo que optimiza la matriz de coste equivocada — un error silencioso: el código correría igual, produciría números, y solo un análisis atento del signo de cada término delataría el fallo. Por eso el notebook fija primero, en prosa y en una tabla 2×2, que:

- La clase positiva (`SeriousDlqin2yrs = 1`) es el cliente que impaga.
- Denegar a un cliente que en realidad iba a pagar es un **Falso Positivo** (coste C_{FP}).
- Conceder a un cliente que en realidad iba a impagar es un **Falso Negativo** (coste C_{FN}).

Y de ahí deriva la fórmula del umbral de decisión bayesiano óptimo:

$$t^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

Esta fórmula es la piedra angular de todo el proyecto: para un clasificador que produce probabilidades bien calibradas, el coste esperado se minimiza denegando exactamente cuando $p(\text{moroso}) > t^*$. Con $C_{FP} = C_{FN} = 1$ da $t^* = 0,5$ (el umbral “de libro” que casi todo el mundo usa sin pensar); con $C_{FP} = 1$, $C_{FN} = 10$ da $t^* = 1/11 \approx 0,091$. El notebook trata este valor teórico como una **hipótesis a contrastar**, no como la respuesta final — en las secciones 4 y 5 se compara sistemáticamente contra el umbral óptimo *empírico* (el que realmente minimiza el coste sobre los datos), y la brecha entre ambos es, en sí misma, una medida de cuán bien calibrado está el modelo.

1.2 Instalación de dependencias

```
# Si falta algún paquete (entorno limpio / Colab), descomentar y ejecutar:  
# %pip install -r requirements.txt
```

[celda 3]. Esta celda está comentada por defecto porque en el entorno de desarrollo ya existe un `venv` con todo instalado (ver `requirements.txt`), y dejarla descomentada añadiría varios minutos a cada re-ejecución sin necesidad. Se deja presente y documentada para que el notebook sea autocontenido si se abre en un entorno limpio (Colab, otra máquina): es la única concesión que hacemos a la portabilidad sin sacrificar la velocidad de iteración local.

1.3 Imports, semilla y generador aleatorio

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from scipy import stats
```

```
SEED = 42
```

```
# Generador moderno de NumPy. Regla del proyecto: nada de np.random.* legacy (estado global);  
# cada sección que necesite aleatoriedad crea su propio default_rng(SEED) para que las celdas
```

```
# sean re-ejecutables en cualquier orden sin alterar resultados.  
rng = np.random.default_rng(SEED)
```

```
pd.options.display.max_columns = None  
sns.set_theme(style="whitegrid", palette="deep")  
plt.rcParams["figure.dpi"] = 110
```

[celda 4]. Tres decisiones merecen explicación:

np.random.default_rng() en vez de np.random.seed() + np.random.randint(). La API legacy de NumPy (`np.random.seed`, `np.random.randint`, etc.) opera sobre un **estado global mutable**: cualquier celda, función o librería de terceros que consuma aleatoriedad legacy en cualquier punto del notebook altera ese estado compartido, así que el resultado de una celda puede depender silenciosamente del orden en que se ejecutaron celdas anteriores — el enemigo número uno de la reproducibilidad en notebooks largos. `default_rng()` crea un objeto generador con su **propio estado encapsulado**: `rng.permutation(...)`, `rng.integers(...)` no tocan ni son tocados por lo que haga cualquier otra parte del código. En este notebook se instancian generadores locales adicionales donde hace falta (por ejemplo `rng_bandit = np.random.default_rng(SEED)` en la sección 10), en vez de reutilizar el `rng` global, precisamente para que cada bloque de aleatoriedad sea reproducible de forma aislada, sin importar si se re-ejecuta esa celda sola o el notebook entero.

pd.options.display.max_columns = None. Con 10-13 columnas, sin este ajuste pandas trunca la vista con ... y hay tablas de diagnóstico (como resumen en la sección 2) que quedarían ilegibles a mitad de auditoría.

sns.set_theme y plt.rcParams["figure.dpi"] = 110. Puramente estético pero deliberado: un DPI de 110 (frente al 72 por defecto de matplotlib) hace que los gráficos —especialmente los `waterfall` de SHAP con texto pequeño en la sección 9— se lean sin pixelado al exportar el notebook a HTML o PDF para la entrega.

Parte 2 — Carga y exploración de datos (EDA)

Esta es, junto con la auditoría, la parte del notebook donde más peso recae en la *reflexión* más que en la mecánica: cada celda de código va seguida de una celda markdown que traduce el número en una decisión. El principio que atraviesa toda la sección es **nunca tocar el preprocesamiento sin haber demostrado primero, con una cifra, que el problema existe**.

2.1 Carga de los tres ficheros

```
df_con = pd.read_csv("cs_construccion.csv")
df_pro = pd.read_csv("cs_produccion.csv")
dicc = pd.read_csv("DataDictionary.csv", sep=";").drop(columns="Unnamed: 0").set_index("Variable Name")
```

```
TARGET = "SeriousDlqin2yrs"
```

```
print(f"Construcción: {df_con.shape[0]:,} filas x {df_con.shape[1]} columnas")
print(f"Producción: {df_pro.shape[0]:,} filas x {df_pro.shape[1]} columnas")
print(f"Mismas columnas y mismo orden: {list(df_con.columns) == list(df_pro.columns)}")
print(f"Target vacío en producción: {df_pro[TARGET].isna().mean():.0%} (es lo que hay que predecir)")
```

```
pd.options.display.max_colwidth = None
dicc
```

[celda 6]. Tres comprobaciones que parecen triviales pero que evitan errores costosos más adelante:

1. `list(df_con.columns) == list(df_pro.columns)` verifica no solo que las columnas coincidan como conjunto, sino que estén **en el mismo orden**. Esto importa porque en la sección 11 se construye el CSV de salida copiando `df_pro` y sobrescribiendo una columna; si el orden difiriera, cualquier operación posicional (en vez de por nombre) rompería el fichero de producción sin lanzar ningún error visible.
2. `df_pro[TARGET].isna().mean()` confirma de forma programática (no solo visual) que el 100% del target de producción está vacío — es decir, que estamos ante un problema de predicción real y no ante datos de validación con etiquetas ocultas en otro sitio.
3. `sep=";"` al leer `DataDictionary.csv` — el fichero usa punto y coma como separador (no coma), algo que solo se descubre inspeccionando el fichero crudo; usar el separador por defecto habría colapsado las cuatro columnas del diccionario en una sola cadena ilegible.

`dicc` se muestra al final de la celda para que el lector del notebook tenga, antes de ver una sola cifra de EDA, la definición de negocio de cada variable a mano (qué significa `DebtRatio`, qué es `NumberOfTime30-59DaysPastDueNotWorse`, etc.) — sin este paso, toda la EDA posterior se leería como aritmética sin significado.

Sigue una celda markdown importante ([celda 7]) que fija una restricción de diseño que condiciona *todo* el resto del proyecto: **producción no tiene columna de identificador de cliente**. El identificador implícito es la posición de la fila dentro del CSV. Esto se traduce en una regla que se repite como comentario en varias funciones del notebook: ninguna transformación puede eliminar, reordenar ni “perder” una fila de producción, porque no hay forma de recuperar después a qué solicitud pertenecía cada predicción.

2.2 Inventario de tipos, nulos y rango de valores

```
resumen = pd.DataFrame({
    "tipo": df_con.dtypes.astype(str),
    "n_nulos": df_con.isna().sum(),
    "%_nulos": (100 * df_con.isna().mean()).round(2),
    "n_unicos": df_con.nunique(),
    "min": df_con.min(),
```

```

    "mediana": df_con.median(),
    "max": df_con.max(),
})
resumen

```

[celda 8]. Una única tabla que concentra siete estadísticos por columna. La elección de columnas del resumen no es genérica: `n_unicos` sirve para detectar variables casi-constantes o, al contrario, con cardinalidad sospechosa de ser continua cuando debería ser discreta; `min` y `max` juntos son los que primero delatan los valores 96/98 en las variables de morosidad (un `max` de 98 en una variable de “número de veces” es ya, a simple vista, una señal de alarma que se investiga en profundidad en el punto 2.4). Construir esta tabla **antes** de cualquier histograma es deliberado: los histogramas con outliers extremos (como los 50.708 puntos de utilización de crédito, ver 2.5) salen ilegibles si no se sabe de antemano que hay que acotar los ejes, y esta tabla es la que avisa de dónde están esos extremos.

2.3 Desbalanceo del target y coste de las estrategias triviales

```

tasa_mora = df_con[TARGET].mean()

fig, ax = plt.subplots(figsize=(4.5, 3))
df_con[TARGET].value_counts(normalize=True).plot(kind="bar", ax=ax, color=["#4C72B0", "#C44E52"])
ax.set_xticklabels(["0 = buen pagador", "1 = moroso"], rotation=0)
ax.set_ylabel("proporción")
ax.set_title(f"Desbalanceo del target: {tasa_mora:.2%} de morosos")
plt.tight_layout()
plt.show()

print("Coste medio de las estrategias triviales (la referencia mínima a batir):")
print(f"  Escenario 1 (FP=1, FN=1) – conceder a todos: {tasa_mora:.4f} | denegar a todos: {1 - tasa_mora:.4f}")
print(f"  Escenario 2 (FP=1, FN=10) – conceder a todos: {10 * tasa_mora:.4f} | denegar a todos: {1 - tasa_mora:.4f}")

```

[celda 10]. Esta celda hace más que dibujar una barra: calcula **el coste de no hacer nada**. Si se concede crédito a todo el mundo (predicción constante 0), todos los errores son falsos negativos, y el coste medio es exactamente `tasa_mora * C_FN`; si se deniega a todo el mundo, todos los errores son falsos positivos y el coste es $(1 - \text{tasa_mora}) * C_{FP}$. Estas dos cifras —0,0668 y 0,9332 en escenario 1; 0,668 y 0,9332 en escenario 2— son el **listón mínimo** que cualquier modelo entrenado tiene que superar para justificar su existencia, y se recuperan literalmente en las secciones 4, 5, 6 y 12 como referencia (`y_test.mean()`, `10*y_test.mean()`). Calcularlas aquí, en la primera celda de EDA, antes de entrenar nada, evita la tentación de reportar solo el coste absoluto del modelo final sin contexto — un coste de 0,335 no dice nada por sí solo; que sea la mitad del coste trivial (0,668) sí.

2.4 Distribuciones y detección de códigos centinela

```

df_con.drop(columns=TARGET).hist(figsize=(14, 10), bins=40)
plt.suptitle("Distribuciones en construcción", y=1.005)
plt.tight_layout()
plt.show()

```

[celda 12]. El histograma conjunto de las 10 variables (excluyendo el target, que ya se visualizó aparte) sirve de “radar” para localizar anomalías antes de dedicar código específico a investigarlas. Es en este gráfico donde aparece, en las tres variables de morosidad, una barra aislada muy a la derecha del resto de la distribución — la pista visual que motiva la siguiente celda.

```

VARS_MORA = ["NumberOfTime30-59DaysPastDueNotWorse",
             "NumberOfTime60-89DaysPastDueNotWorse",
             "NumberOfTimes90DaysLate"]

for c in VARS_MORA:
    print(f"{c} – valores más altos:")
    print(df_con[c].value_counts().sort_index().tail(4).to_string(), "\n")

mask_cent = (df_con[VARS_MORA] >= 96).any(axis=1)
en_las_tres = (df_con[VARS_MORA] >= 96).all(axis=1)
print(f"Filas con algún 96/98: {mask_cent.sum()} | con 96/98 en las TRES variables a la vez: {en_las_tres.sum()}")
print(f"Tasa de mora en esas filas: {df_con.loc[mask_cent, TARGET].mean():.1%} (global: {tasa_mora:.1%})")
print(f"En producción hay {(df_pro[VARS_MORA] >= 96).any(axis=1).sum()} filas con estos códigos (también hay que predecir")

```

[celda 14]. El paso clave de esta celda es la comparación entre `mask_cent` (`.any(axis=1)`, al menos una de las tres variables tiene 96/98) y `en_las_tres` (`.all(axis=1)`, las tres a la vez). Que ambas máscaras den el mismo número (188) es la prueba estadística de que 96 y 98 no son recuentos reales sino un **código administrativo que se propaga a las tres columnas simultáneamente** — la hipótesis alternativa (que sean, por azar, historiales genuinamente malos en cada una de las tres variables por separado) queda descartada porque sería estadísticamente imposible que las tres coincidieran en el mismo subconjunto exacto de filas por casualidad. Esta sola comparación de dos máscaras booleanas es la evidencia que después justifica, en la sección 3, colapsar las tres variables contaminadas en un único `flag_mora_extrema` en vez de tratarlas como tres señales independientes.

La última línea comprueba el mismo patrón en `df_pro`: encontrar 81 filas con el código en producción es lo que descarta la opción, tentadora por simplicidad, de simplemente eliminar esas 188 filas de construcción como hace el notebook de clase — no se puede aplicar la misma eliminación a producción sin perder 81 predicciones obligatorias.

2.5 Missingness informativo

```

print(f"Mora con MonthlyIncome informado: {df_con.loc[df_con['MonthlyIncome'].notna(), TARGET].mean():.2%}")
print(f"Mora con MonthlyIncome ausente: {df_con.loc[df_con['MonthlyIncome'].isna(), TARGET].mean():.2%}")
print()
print(f"Nulos MonthlyIncome – construcción: {df_con['MonthlyIncome'].isna().mean():.1%} | producción: {df_pro['MonthlyIncome'].isna().mean():.1%}")
print(f"Nulos NumberOfDependents – construcción: {df_con['NumberOfDependents'].isna().mean():.1%} | producción: {df_pro['NumberOfDependents'].isna().mean():.1%}")

```

[celda 16]. La comparación central de esta celda es `df_con.loc[...isna()..., TARGET].mean()` frente a `df_con.loc[...notna()..., TARGET].mean()`: son las tasas de mora condicionadas a que el ingreso esté o no informado. Si el missingness fuera **completamente al azar** (MCAR, en la taxonomía de Little y Rubin), ambas tasas deberían ser estadísticamente indistinguibles; que salgan 5,5% y 7,0% respectivamente demuestra que no lo es, y que por tanto la variable “está ausente” contiene información sobre la probabilidad de mora independiente del valor del ingreso en sí. Esta es la justificación técnica precisa (no solo intuitiva) para crear un `flag_sin_ingreso` explícito en vez de imputar silenciosamente la mediana: imputar sin flag haría indistinguibles, a ojos del modelo, a un cliente con ingreso realmente mediano de otro cuyo ingreso es desconocido — perdiendo justo la señal que esta celda demuestra que existe.

2.6 Utilización de crédito: señal fuerte y probable error de unidad

```

bandas = pd.cut(df_con["RevolvingUtilizationOfUnsecuredLines"],
               bins=[-0.001, 0.5, 1, 10, np.inf],
               labels=["≤ 0,5", "(0,5 – 1]", "(1 – 10]", "> 10"])
tabla_util = df_con.groupby(bandas, observed=True)[TARGET].agg(n="size", tasa_mora="mean")

```



```
tabla_util["tasa_mora"] = (100 * tabla_util["tasa_mora"]).round(1)
tabla_util
```

[celda 18]. `RevolvingUtilizationOfUnsecuredLines` es conceptualmente un ratio (saldo usado / límite de crédito) y por tanto debería estar acotado en $[0, \sim 1]$, con algo de margen si el cliente rebasa su límite. Cortar la variable en cuatro bandas con `pd.cut` y calcular la tasa de mora `groupby` por banda revela un patrón **no monótono**: el riesgo sube progresivamente hasta la banda $(1, 10]$ (39,5% de mora) y luego **cae de golpe** en la banda > 10 (6,8%, la tasa base). Un ratio de utilización real de “50 veces el límite” sería el perfil más arriesgado imaginable, no uno neutro — la caída abrupta es la firma estadística de un **error de captura de datos** (probablemente saldos absolutos en dólares grabados en la columna que debía llevar el ratio), no de un fenómeno de negocio real. Esta única tabla `groupby` sostiene tres decisiones posteriores: (a) no eliminar esos 176 casos, porque romper la fila 45.000-ésima de producción no es una opción; (b) para los árboles, no hace falta tratamiento especial porque particionan el espacio de la variable en umbrales, así que aíslan la banda > 10 sin que “contamine” el aprendizaje sobre el resto; (c) para la regresión logística sí hace falta winsorizar, porque una combinación lineal de esa variable sin acotar dispararía coeficientes o (tras estandarizar) comprimiría toda la señal útil en un rango minúsculo frente al outlier.

2.7 Duplicados exactos y edades imposibles

```
print(f"Duplicados exactos (las 11 columnas idénticas): {df_con.duplicated().sum()} filas sobrantes "
      f"{df_con.duplicated(keep=False).sum()} filas implicadas")
print(f"Filas con age = 0: {(df_con['age'] == 0).sum()} en construcción, {(df_pro['age'] == 0).sum()} en producción")
print(f"Edad mínima real: {df_con.loc[df_con['age'] > 0, 'age'].min()} años (coherente: los menores no pueden solicitar c
```

[celda 20]. `df_con.duplicated()` (sin argumentos) marca como duplicada toda fila cuyas 11 columnas —las 10 *features* más el *target*— coincidan exactamente con una fila anterior; con `keep=False` se marcan **todas** las copias, incluida la primera, lo que permite reportar tanto “cuántas filas sobran” (331, útil para saber cuánto se va a reducir el dataset) como “cuántas filas están implicadas en total” (547, útil para dimensionar el riesgo si esos duplicados cayeran repartidos entre entrenamiento y test). La razón de auditar duplicados exactos —no solo casi-duplicados— es concreta: si una misma fila aparece en el conjunto de entrenamiento y en el de test tras un `train_test_split` aleatorio, el modelo “ve la respuesta” de ese caso de test durante el entrenamiento, inflando artificialmente cualquier métrica de generalización. La comprobación de `age = 0` usa el mismo patrón (contar en construcción y en producción) que ya apareció con los códigos 96/98: producción no tiene ninguna fila con edad 0, así que eliminar esa única fila de construcción no genera la asimetría de tratamiento que sí había con los códigos de morosidad.

2.8 Matriz de correlaciones y el artefacto de los códigos centinela

```
fig, ax = plt.subplots(figsize=(9, 7))
sns.heatmap(df_con.corr(), cmap="bwr", vmin=-1, vmax=1, annot=True, fmt=".2f",
            ax=ax, cbar_kws={"shrink": 0.8})
ax.set_title("Matriz de correlaciones (construcción)")
plt.tight_layout()
plt.show()

print("Correlación entre las tres variables de morosidad CON códigos 96/98:")
print(df_con[VARS_MORA].corr().round(2).to_string())
print("\nY SIN los códigos 96/98:")
print(df_con.loc[~mask_cent, VARS_MORA].corr().round(2).to_string())
```

[celda 21]. El heatmap general (con `vmin=-1`, `vmax=1` fijado explícitamente para que el mapa de color sea comparable entre ejecuciones y no se reescale según el rango real de datos) muestra correlaciones de 0,98-0,99 entre las tres variables de morosidad — a primera vista, una multicolinealidad severísima que

podría preocupar en un modelo lineal. La segunda mitad de la celda diseña un experimento controlado para diagnosticar la causa: recalcula la correlación **excluyendo** las filas marcadas por `mask_cent` (la máscara de códigos 96/98 ya calculada en la celda 14) y la correlación cae a 0,27-0,33. La conclusión no es que las tres variables sean redundantes en general, sino que **188 filas con un valor idéntico y atípico en las tres columnas** son las que arrastran el coeficiente de correlación de todo el dataset hacia 1 — un ejemplo de libro de cómo un pequeño número de puntos extremos puede dominar un estadístico calculado sobre 105.000 filas. Esta es la segunda pieza de evidencia (junto con la tasa de mora del 54,3%) que justifica la recodificación de la sección 3: si no se colapsan en un flag, el modelo (y, peor aún, el análisis SHAP de la sección 9) repartiría la importancia de “tener el código especial” entre tres variables casi idénticas, diluyendo su interpretación.

2.9 Ausencia de drift entre construcción y producción

```
feats = [c for c in df_con.columns if c != TARGET]
drift = pd.DataFrame({
    "mediana_construcción": df_con[feats].median(),
    "mediana_producción": df_pro[feats].median(),
    "media_construcción": df_con[feats].mean(),
    "media_producción": df_pro[feats].mean(),
    "KS": [stats.ks_2samp(df_con[c].dropna(), df_pro[c].dropna()).statistic for c in feats],
}).round(4)
drift
```

[celda 23]. El test de Kolmogorov-Smirnov de dos muestras (`stats.ks_2samp`) compara, variable a variable, la función de distribución empírica de construcción contra la de producción, y devuelve el estadístico D (la distancia máxima vertical entre ambas curvas acumuladas): valores cercanos a 0 indican que las dos muestras son indistinguibles; valores grandes, que hay *drift*. Ejecutarlo en un bucle sobre las 10 variables da un valor máximo de 0,0073 (en `age`), muy por debajo de cualquier umbral de alarma habitual (0,1-0,2). Esta celda cierra el argumento metodológico que se explota en toda la segunda mitad del notebook: si construcción y producción son estadísticamente la misma población, **el coste medido en el conjunto de test interno es un estimador fiable del coste que obtendrán los ficheros de producción**, sin necesidad de técnicas de adaptación de dominio ni de re-ponderar por importancia (*importance weighting*). Se calcula aquí, en la EDA, y no se repite el chequeo en la sección 11: haberlo demostrado una vez con rigor estadístico basta.

Parte 3 — Preprocesamiento

3.1 La función `prepara_features`: la única puerta de entrada de datos

`CAP_MORA = 20` # por encima del máximo real observado (12/8/15), por debajo de los códigos 96/98

```
def prepara_features(df):
    """Ingeniería de variables común a todos los modelos y a producción.

    Regla de oro: NO elimina, imputa-por-descarte ni reordena filas – producción
    (45.000 solicitudes sin identificador, la posición es el id) debe salir íntegra.
    """
    X = df.drop(columns=TARGET).copy()
    X["flag_mora_extrema"] = (X[VARS_MORA] >= 96).any(axis=1).astype(int)
    X[VARS_MORA] = X[VARS_MORA].clip(upper=CAP_MORA)
    X["flag_sin_ingreso"] = X["MonthlyIncome"].isna().astype(int)
    X["flag_sin_dependientes"] = X["NumberOfDependents"].isna().astype(int)
    return X
```

[celda 26]. Esta función de ocho líneas es, en un sentido muy literal, el contrato entre la EDA (sección 2) y todo lo que viene después: cada una de sus cuatro transformaciones responde a un hallazgo concreto de la sección anterior, y ninguna transformación de la sección 2 que no esté aquí llega a los modelos.

- `(X[VARS_MORA] >= 96).any(axis=1).astype(int)` crea el flag binario que resume el hallazgo de la celda 14: la conjunción exacta con `.all()` demostró que el código aparece siempre en las tres columnas a la vez, así que basta comprobar si aparece en *alguna* (`.any`) para detectarlo sin ambigüedad, y el resultado se convierte a entero porque LightGBM y la logística esperan variables numéricas, no booleanas, en el DataFrame que reciben.
- `.clip(upper=CAP_MORA)` con `CAP_MORA = 20` sustituye los códigos 96/98 (y cualquier valor intermedio anómalo) por un tope de 20, elegido deliberadamente **por encima** del máximo verosímil observado (12, 8 y 15 en las tres variables respectivamente, ver sección 2.4) pero **muy por debajo** de 96. El propósito de capar en vez de, por ejemplo, sustituir por la mediana, es que el modelo siga viendo “muchísimos impagos” como una magnitud grande y ordenada, coherente con el resto de la escala, sin que el valor 98 se comporte como un salto discontinuo que distorsione las particiones de los árboles o los coeficientes de la logística.
- `flag_sin_ingreso` y `flag_sin_dependientes` materializan la conclusión estadística de la celda 16 (que la ausencia de `MonthlyIncome` es informativa) como una columna que el modelo puede usar directamente, en vez de dejar que la información se pierda al imputar.
- El *docstring* de la función no es decorativo: enuncia la restricción de integridad de filas como una regla de diseño, y esa misma regla se verifica más abajo con un `assert` explícito.

Es importante notar lo que la función **no** hace: no imputa `MonthlyIncome` ni `NumberOfDependents` (los deja como `NaN`), y no toca `RevolvingUtilizationOfUnsecuredLines` pese al hallazgo de la sección 2.5. Ambas omisiones son intencionadas y se explican en la tabla de decisiones de la siguiente subsección: LightGBM enruta de forma nativa los valores ausentes durante el entrenamiento (aprende, en cada nodo, hacia qué rama enviar los `NaN` según cuál minimiza la pérdida), así que imputar antes de dárselos al árbol sería *destruir* información que el algoritmo sabe aprovechar mejor por sí mismo. El tratamiento de la utilización de crédito y de los nulos para la regresión logística se hace aparte, en un pipeline específico (ver 3.4), precisamente porque ese modelo sí lo necesita y el de árboles no.

3.2 Limpieza de filas: solo en construcción, nunca en producción

```
# Limpieza SÓLO de construcción (filas): duplicados y la edad imposible
df_limpio = df_con.drop_duplicates()
```

```

df_limpio = df_limpio[df_limpio["age"] > 0]
print(f"Construcción: {len(df_con):,} → {len(df_limpio):,} filas "
      f"(-{len(df_con) - len(df_limpio)}: duplicados y edad 0)")

X_full = prepara_features(df_limpio)
y_full = df_limpio[TARGET]

X_prod = prepara_features(df_pro)
assert len(X_prod) == len(df_pro) == 45_000, "Producción debe salir íntegra"
print(f"Producción: {len(X_prod):,} filas conservadas (íntegra) con {X_prod.shape[1]} features")
X_prod.head(3)

```

[celda 27]. Esta celda separa deliberadamente en dos rutas distintas las dos operaciones de limpieza que sí eliminan filas (`drop_duplicates()` y el filtro `age > 0`) de la función `prepara_features`, que nunca las toca. La razón estructural es que estas dos limpiezas **solo tienen sentido en construcción**: los duplicados exactos son un riesgo de fuga de información entre entrenamiento y test (sección 2.7) que no existe en un conjunto de predicción que no se va a partir en folds, y la edad 0 es un valor inválido que no aparece en producción (ya verificado en la celda 20). El `assert len(X_prod) == len(df_pro) == 45_000` no es un chequeo cosmético: es la comprobación programática de la restricción de integridad enunciada en el docstring de `prepara_features`, colocada justo después de la única línea que podría, por error de copia y pega, haber aplicado el mismo filtrado a producción. Si algún cambio futuro introdujera sin querer un `.dropna()` o un filtro sobre `X_prod`, este `assert` fallaría de inmediato en vez de dejar que el error se propagara silenciosamente hasta la celda 84, donde se escriben los CSV finales.

3.3 Esquema de validación: por qué 80/20 más 5-fold OOF

```

from sklearn.model_selection import train_test_split

X_dev, X_test, y_dev, y_test = train_test_split(
    X_full, y_full, test_size=0.20, stratify=y_full, random_state=SEED)

print(f"Desarrollo: {len(X_dev):,} filas ({y_dev.mean():.2%} morosos)")
print(f"Test final: {len(X_test):,} filas ({y_test.mean():.2%} morosos)")

```

[celda 29]. `stratify=y_full` asegura que la proporción de morosos (6,70%) se mantenga igual en desarrollo y en test, evitando que un split aleatorio “de mala suerte” concentre desproporcionadamente la (escasa) clase positiva en uno de los dos conjuntos, lo que sesgaría tanto la estimación de hiperparámetros como la del coste final. La celda markdown que la precede ([celda 28]) explica el porqué de este split en el contexto más amplio del proyecto: la comparación de modelos y la elección del umbral de decisión ocurren enteramente **dentro** de `X_dev/y_dev`, usando validación cruzada estratificada de 5 folds y evaluando siempre sobre predicciones *out-of-fold* (OOF) — es decir, la predicción de cada fila la produce un modelo que no la vio durante su entrenamiento, aunque todas las filas de desarrollo participen en la comparación. `X_test/y_test` permanece completamente al margen de esa fase y solo se usa una vez, al final, para reportar el coste. Esta separación es la que impide el error metodológico más común en optimización cost-sensitive: elegir el umbral que minimiza el coste sobre el mismo conjunto que después se usa para reportarlo, lo que sistemáticamente subestima el coste real (el umbral “aprende” el ruido específico de esa muestra).

3.4 El pipeline de la regresión logística: winsorización, imputación y escalado

```

# Pipeline de la regresión logística (la parte que SÍ necesita imputación, recorte y escala)
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

```

```

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

VARS_SESGADAS = ["RevolvingUtilizationOfUnsecuredLines", "DebtRatio", "MonthlyIncome"]

class WinsorizadorLog(BaseEstimator, TransformerMixin):
    """Recorta la cola derecha al percentil p (aprendido en train) y aplica log1p.

    Neutraliza tanto las colas largas legítimas como los errores de unidad de $2.5,
    a los que la logística es muy sensible (los árboles no lo necesitan).
    """
    def __init__(self, columnas, p=99.0):
        self.columnas = columnas
        self.p = p

    def fit(self, X, y=None):
        self.feature_names_in_ = np.asarray(X.columns, dtype=object)
        self.limite_ = {c: np.percentile(X[c].dropna(), self.p) for c in self.columnas}
        return self

    def transform(self, X):
        X = X.copy()
        for c, lim in self.limite_.items():
            X[c] = np.log1p(X[c].clip(upper=lim))
        return X

    def get_feature_names_out(self, input_features=None):
        return self.feature_names_in_ if input_features is None else np.asarray(input_features, dtype=object)

def crea_pipe_logistica(**kw_logistica):
    pipe = Pipeline([
        ("winsor", WinsorizadorLog(VARS_SESGADAS)),
        ("imputa", SimpleImputer(strategy="median")),
        ("escala", StandardScaler()),
        ("modelo", LogisticRegression(max_iter=2000, random_state=SEED, **kw_logistica)),
    ])
    pipe.set_output(transform="pandas") # conserva nombres de columnas para leer coeficientes
    return pipe

crea_pipe_logistica()

```

[celda 30]. Este es el bloque de código más denso en decisiones de diseño de todo el preprocesamiento, así que merece un desglose pieza por pieza.

Por qué un transformador sklearn personalizado y no una función suelta. WinsorizadorLog hereda de BaseEstimator y TransformerMixin, que son las clases base que sklearn exige para que un objeto se comporte como cualquier otro paso de un Pipeline: exponer fit(X, y) y transform(X), y —vía TransformerMixin— obtener gratis un fit_transform. La razón de no resolver esto con una función que simplemente recorta y aplica logaritmo es la fuga de información: el percentil 99 que define el corte de cada variable **debe aprenderse solo con datos de entrenamiento** (fit) y aplicarse igual, sin recalcularlo, sobre validación, test y producción (transform). Envolver esta lógica en un Pipeline de sklearn hace que cross_val_predict (usado extensamente en las secciones 4 y 5) recalcule automáticamente los límites de winsorización en cada uno de los 5 folds, evitando que la información de los folds de validación se filtre en el cálculo del percentil de

corte — un error sutil pero común si se winsoriza el dataset completo antes de partir en folds.

fit guarda feature_names_in_ y los límites por columna; transform aplica clip + log1p. $\log1p(x) = \log(1+x)$ en vez de $\log(x)$ evita $-\inf$ cuando $x = 0$ (que ocurre, por ejemplo, en `DebtRatio` para muchos clientes). Aplicar primero el recorte (`clip(upper=lim)`) y después el logaritmo, y no al revés, importa: recortar en escala logarítmica desplazaría el punto de corte de forma no lineal y ya no correspondería exactamente al percentil 99 de la variable original.

get_feature_names_out. Este método no está aquí por estética: `sklearn 1.8` exige que cualquier transformador dentro de un `Pipeline` con `set_output(transform="pandas")` sepa declarar los nombres de las columnas que produce; sin este método, la llamada a `pipe.set_output(transform="pandas")` falla en tiempo de ejecución con un `ValueError` explícito ("Unable to configure output ... because set_output is not available") — un error que se encontró y resolvió durante el desarrollo de este mismo notebook. Como `WinsorizadorLog` no añade ni quita columnas, el método simplemente devuelve los nombres que vio en `fit`.

set_output(transform="pandas") en el pipeline completo. Sin esta línea, cada paso del pipeline (`SimpleImputer`, `StandardScaler`) devolvería arrays de `NumPy` desnudos, perdiendo los nombres de columna en el camino. Mantenerlos como `DataFrame` en todo momento es lo que permite, más adelante, inspeccionar los coeficientes de `LogisticRegression` asociados a cada variable por nombre, en vez de por posición — algo que se vuelve indispensable a la hora de auditar el modelo.

Por qué VARS_SESGADAS son justo esas tres. `RevolvingUtilizationOfUnsecuredLines` por el error de unidad de la sección 2.5, `DebtRatio` porque el inventario de la celda 8 mostró valores por encima de 10 (un ratio deuda/ingreso de 1000% es también sospechoso de ser un error de captura), y `MonthlyIncome` por su cola larga natural (algunos ingresos declarados por encima de 1 millón). El resto de variables (recuentos de líneas de crédito, edad, número de dependientes) no se winsorizan: son enteros pequeños y acotados, sin outliers patológicos.

Por qué la logística necesita todo esto y LightGBM no. Es el contraste que se repite en toda la tabla de decisiones de la sección: un modelo lineal calcula una combinación ponderada de las variables de entrada, así que su ajuste es sensible a la escala relativa entre ellas (`StandardScaler`) y a la presencia de valores extremos que dominen el gradiente durante la optimización (winsorización); un árbol de decisión particiona el espacio comparando cada variable contra un umbral, una operación invariante a cualquier transformación monótona (log, escalado) y robusta a valores extremos aislados, porque un único outlier cae en una sola hoja sin distorsionar el resto de las particiones.

3.5 Interpretación de la sección 3

La celda markdown de cierre ([celda 31]) resume el principio que gobierna toda la sección: `prepara_features` es la única puerta de entrada de datos a los modelos, y garantiza por construcción — con el `assert` de la celda 27 — que producción sale íntegra. La existencia de **dos** preparaciones distintas (mínima para los árboles, vía `prepara_features`; completa para la logística, vía el `Pipeline` de la celda 30) no es una inconsistencia sino una consecuencia directa de que cada familia de modelos tiene un sesgo inductivo distinto: forzar a los árboles a pasar por winsorización y escalado no les haría ningún bien (son invariantes a ello) y sí les quitaría la capacidad de `LightGBM` de aprender de los patrones de nulidad de forma nativa.

Parte 4 — Modelado, escenario 1 ($C_{FP} = C_{FN} = 1$)

4.1 La maquinaria de coste: `coste_medio`, `curva_coste`, `umbral_optimo`

```
# --- Helper: maquinaria de coste y umbral (núcleo de negocio del proyecto) ---

def coste_medio(y_true, y_pred, c_fp, c_fn):
    """Coste medio por solicitud según la matriz {FP: c_fp, FN: c_fn}."""
    y_true = np.asarray(y_true); y_pred = np.asarray(y_pred)
    fp = np.sum((y_pred == 1) & (y_true == 0)) # denegado a buen pagador
    fn = np.sum((y_pred == 0) & (y_true == 1)) # concedido a moroso
    return (c_fp * fp + c_fn * fn) / len(y_true)

def curva_coste(y_true, proba, c_fp, c_fn):
    """Coste medio en TODOS los umbrales candidatos (exacto y vectorizado).

    Con umbral t denegamos si p > t. Ordenando por probabilidad:
    FN(t) = morosos con p <= t; FP(t) = buenos con p > t. Ambas son acumulados.
    """
    y = np.asarray(y_true)
    orden = np.argsort(proba, kind="mergesort")
    p_ord, y_ord = np.asarray(proba)[orden], y[orden]
    fn_acum = np.cumsum(y_ord)
    negativos = 1 - y_ord
    fp_restante = negativos.sum() - np.cumsum(negativos)
    idx = np.searchsorted(p_ord, np.unique(p_ord), side="right") - 1
    umbrales = np.concatenate([-np.inf, p_ord[idx]]) # -inf = denegar a todos
    costes = np.concatenate([c_fp * negativos.sum() / len(y),
                              (c_fn * fn_acum[idx] + c_fp * fp_restante[idx]) / len(y)])
    return umbrales, costes

def umbral_optimo(y_true, proba, c_fp, c_fn):
    """Umbral de coste mínimo; se devuelve el punto medio con el siguiente candidato
    para no dejar la frontera exactamente sobre una observación de entrenamiento."""
    t, c = curva_coste(y_true, proba, c_fp, c_fn)
    i = int(np.argmin(c))
    t_opt = (t[i] + t[i + 1]) / 2 if (i + 1 < len(t) and np.isfinite(t[i])) else t[i]
    return t_opt, c[i]

# sanity check contra la estrategia trivial
assert np.isclose(coste_medio(y_dev, np.zeros(len(y_dev)), 1, 1), y_dev.mean())
print("Maquinaria de coste lista")
```

[celda 33]. Estas tres funciones son, en un sentido literal, el “motor de negocio” que decide qué modelo gana y dónde se pone el punto de corte en todo el resto del notebook, así que merecen una explicación algorítmica completa.

`coste_medio` es la más simple: cuenta falsos positivos y falsos negativos con máscaras booleanas vectorizadas y aplica la definición de la sección 1. Nótese que ambas conversiones a `np.asarray` al principio son defensivas: `y_pred` y `y_true` pueden llegar como `pandas.Series` (con índices que no necesariamente coinciden entre sí) o como arrays de NumPy; comparar `Series` con índices distintos elemento a elemento produciría resultados alineados por índice en vez de por posición, un error silencioso particularmente peligroso. Convertir a array fuerza la comparación posicional, que es la semánticamente correcta aquí.

curva_coste es la función más sofisticada del notebook, porque resuelve un problema que ingenuamente se abordaría con un bucle sobre una rejilla de umbrales (`for t in np.linspace(0, 1, 1000)`) de forma **exacta y sin rejilla**, en tiempo $O(n \log n)$ en vez de $O(n \cdot m)$ para m umbrales candidatos. La idea es la siguiente: ordenando todas las probabilidades predichas de menor a mayor, mover el umbral de decisión de un candidato al siguiente equivale a cambiar de bando, exactamente, a una observación cada vez. Así, en vez de recalculer FP y FN desde cero para cada umbral, se calculan sus dos series acumuladas mediante `np.cumsum`:

- `fn_acum = np.cumsum(y_ord)`: en el umbral situado justo después de la observación ordenada en la posición i , el número de falsos negativos es la suma de los positivos (`y_ord == 1`) entre las posiciones 0 e i — porque esos son los morosos con probabilidad por debajo del umbral, a quienes por tanto se concede crédito por error.
- `fp_restante = negativos.sum() - np.cumsum(negativos)`: el número de falsos positivos en ese mismo punto es el total de buenos pagadores **menos** los que ya han quedado “por debajo” del umbral acumulado — es decir, los buenos pagadores que quedan por encima y por tanto se deniegan por error.

Ambas se calculan con una única pasada de `cumsum` sobre el array ordenado, y el coste en cada umbral candidato se obtiene con una combinación lineal vectorizada de ambas series — sin bucles en Python. `np.searchsorted(p_ord, np.unique(p_ord), side="right") - 1` localiza, para cada valor único de probabilidad, el índice del *último* elemento ordenado con ese valor exacto (relevante cuando hay probabilidades repetidas, algo habitual en un gradient boosting con árboles poco profundos, donde muchas observaciones caen en la misma hoja y por tanto reciben idéntica probabilidad). Finalmente se antepone el candidato `-inf` (denegar a todos, `coste_c_fp * proporción_de_buenos`) para que la curva cubra también ese extremo.

umbral_optimo llama a `curva_coste`, localiza el índice de coste mínimo con `np.argmin`, y devuelve el **punto medio** entre ese candidato y el siguiente en vez del candidato exacto. Este detalle, que podría parecer superfluo, evita un problema real: si el umbral óptimo se fija exactamente sobre el valor de probabilidad de una observación de entrenamiento, esa observación queda justo en la frontera de decisión, y pequeñas variaciones numéricas (por ejemplo, al recalculer las probabilidades en un reentrenamiento con datos ligeramente distintos) podrían cambiar su clasificación de forma inestable. Tomar el punto medio entre dos candidatos consecutivos sitúa la frontera en una región sin datos de entrenamiento exactamente encima.

El assert final es una prueba unitaria mínima pero real: comprueba que predecir “0 para todos” (`np.zeros`) con `C_FP = C_FN = 1` produce, por la función genérica `coste_medio`, el mismo número que la fórmula cerrada `y_dev.mean()` calculada a mano en la sección 2.3. Si `coste_medio` tuviera un error de signo o de conteo, este `assert` fallaría de inmediato, antes de que ese error se propagara silenciosamente a docenas de celdas posteriores que dependen de esta función.

4.2 Comparativa de candidatos por validación cruzada

```
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.metrics import roc_auc_score, brier_score_loss
import lightgbm as lgb
from xgboost import XGBClassifier

def crea_lgbm(**extra):
    params = dict(n_estimators=300, learning_rate=0.10, num_leaves=15, min_child_samples=60,
                  subsample=0.8, subsample_freq=1, colsample_bytree=0.8, reg_lambda=1.0,
                  random_state=SEED, n_jobs=-1, verbosity=-1)
    params.update(extra)
    return lgb.LGBMClassifier(**params)

candidatos = {
    "LightGBM compacto (300x15)": crea_lgbm(),
    "LightGBM medio (500x31)": crea_lgbm(n_estimators=500, num_leaves=31, learning_rate=0.05),
```



```

"LightGBM grande (800x63)": crea_lgbm(n_estimators=800, num_leaves=63, learning_rate=0.03),
"XGBoost (500, prof 6)": XGBClassifier(n_estimators=500, learning_rate=0.05, max_depth=6,
                                     subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0,
                                     tree_method="hist", random_state=SEED, n_jobs=-1,
                                     eval_metric="logloss"),
"Regresión logística L2": crea_pipe_logistica(),
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
oof = {}
filas = []
for nombre, modelo in candidatos.items():
    proba = cross_val_predict(modelo, X_dev, y_dev, cv=cv, method="predict_proba")[:, 1]
    oof[nombre] = proba
    t1_, c1_ = umbral_optimo(y_dev, proba, 1, 1)
    t2_, c2_ = umbral_optimo(y_dev, proba, 1, 10)
    filas.append({"modelo": nombre,
                 "AUC": roc_auc_score(y_dev, proba),
                 "Brier": brier_score_loss(y_dev, proba),
                 "umbral*_E1": t1_, "coste_00F_E1": c1_,
                 "umbral*_E2": t2_, "coste_00F_E2": c2_})
tabla_cv = pd.DataFrame(filas).set_index("modelo").round(4)
tabla_cv

```

[celda 34]. Varias decisiones de diseño concentradas en pocas líneas:

crea_lgbm como fábrica de modelos con parámetros por defecto sobreescribibles (params.update(extra)). Esto permite reutilizar exactamente la misma función, con la misma regularización base, en las tres configuraciones de tamaño (compacto, medio, grande) y más adelante, en la sección 5, para la variante `scale_pos_weight=10` — sin duplicar la lista de hiperparámetros cuatro veces y arriesgarse a que diverjan por un descuido de copia y pega. Los valores de regularización elegidos (`min_child_samples=60`, `subsample=0.8`, `colsample_bytree=0.8`, `reg_lambda=1.0`) son conservadores a propósito: con solo 13 variables y 83.734 filas de desarrollo, un gradient boosting sin restricciones tiende a sobreajustar, especialmente en la cola de probabilidades altas donde hay pocos ejemplos positivos.

Por qué tres tamaños de LightGBM y no uno solo. Explorar compacto/medio/grande no es una búsqueda exhaustiva de hiperparámetros (no hay `GridSearchCV` ni `Optuna` aquí) sino una comprobación deliberadamente simple de si “más capacidad ayuda”: con un problema tabular de baja dimensión como este, es una hipótesis razonable que un modelo más grande simplemente memorice ruido en vez de señal adicional, y los resultados (sección 4.3) confirman precisamente eso.

StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED) reutilizado como objeto cv para todos los candidatos. Fijar el objeto `cv` una sola vez y pasarlo a cada llamada de `cross_val_predict` garantiza que **los cinco modelos se comparan sobre exactamente los mismos folds** — si cada modelo generara su propia partición aleatoria, las diferencias de coste observadas podrían deberse en parte a que un modelo tuvo, por azar, una partición más favorable, en vez de a una diferencia real de capacidad predictiva.

cross_val_predict(..., method="predict_proba")[:, 1] es la pieza que produce las predicciones *out-of-fold*: para cada fila de `X_dev`, la probabilidad devuelta procede del modelo entrenado en los otros 4 folds, nunca del fold que contiene esa fila. Es este array `proba` el que se usa tanto para calcular AUC y Brier como para buscar el umbral óptimo con `umbral_optimo` — así, ni la comparación de modelos ni la elección de umbral incurrir en el sesgo optimista de evaluar sobre datos ya vistos en entrenamiento.

roc_auc_score y brier_score_loss como métricas complementarias al coste. El AUC mide la capacidad de *ordenar* correctamente clientes por riesgo, con independencia de cualquier umbral; el Brier score (`mean((p`

– $y)^2$) mide la **calidad de la calibración** — cuánto se parecen las probabilidades predichas a frecuencias reales. Ambas se calculan aquí no porque hagan falta para decidir el umbral (eso ya lo resuelve `umbral_optimo` directamente sobre el coste), sino porque son las que permiten, en la sección 5, diagnosticar por qué el modelo con `scale_pos_weight=10` empatara en coste pero duplica su Brier: una propiedad invisible si solo se mirara el coste.

Calcular de una vez `umbral*_E1` y `coste_00F_E1` junto con `umbral*_E2` y `coste_00F_E2`, para los cinco candidatos, en la misma tabla. Esto es deliberado: permite, en la siguiente celda, justificar la elección del modelo ganador con evidencia de que gana en **ambos** escenarios simultáneamente, no solo en el que se está a punto de resolver.

4.3 Elección del modelo ganador y curva de coste del escenario 1

```
NOMBRE_GANADOR = "LightGBM compacto (300x15)"
oof_ganador = oof[NOMBRE_GANADOR]

C_FP_1, C_FN_1 = 1, 1
t1_dev, c1_oof = umbral_optimo(y_dev, oof_ganador, C_FP_1, C_FN_1)

t_grid, c_grid = curva_coste(y_dev, oof_ganador, C_FP_1, C_FN_1)
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_grid[1:], c_grid[1:], lw=1)
ax.axvline(t1_dev, color="crimson", ls="--", label=f"umbral óptimo 00F = {t1_dev:.3f}")
ax.axvline(0.5, color="gray", ls=":", label="umbral teórico t* = 0,5")
ax.axhline(y_dev.mean(), color="green", ls=":", label=f"trivial 'conceder a todos' = {y_dev.mean():.4f}")
ax.set_xlabel("umbral de decisión sobre p(moroso)")
ax.set_ylabel("coste medio 00F")
ax.set_title("Escenario 1: coste vs umbral (00F desarrollo)")
ax.set_ylim(0.05, 0.11)
ax.legend()
plt.tight_layout(); plt.show()

print(f"Umbral óptimo 00F: {t1_dev:.4f} → coste 00F {c1_oof:.4f} (en t=0,5: "
      f"{coste_medio(y_dev, (oof_ganador > 0.5).astype(int), 1, 1):.4f})")
```

[celda 36]. `NOMBRE_GANADOR` se fija literalmente (“a mano”) tras leer la `tabla_cv` de la celda anterior — una elección editorial deliberada del notebook para dejar constancia explícita de la decisión, en vez de automatizarla con un `tabla_cv["coste_00F_E1"].idxmin()` que ocultaría el razonamiento humano detrás de la elección. `t_grid[1:]` y `c_grid[1:]` descartan el primer elemento (el candidato `-inf`, es decir “denegar a todos”) solo para el dibujo, porque incluirlo distorsionaría la escala del eje X sin aportar información visual relevante (ya lo cubre la línea horizontal verde de la estrategia trivial). Fijar `ax.set_ylim(0.05, 0.11)` explícitamente, en vez de dejar que matplotlib autoescale, es necesario porque el coste en el extremo “denegar a todos” (0,933) desbordaría cualquier escala útil para ver el mínimo real de la curva, que se mueve en un rango mucho más estrecho.

4.4 Evaluación única en test

```
# Evaluación única en test del escenario 1 (modelo reentrenado en desarrollo, umbral congelado)
modelo_dev = crea_lgbm().fit(X_dev, y_dev)
proba_test = modelo_dev.predict_proba(X_test)[: , 1]

pred1_test = (proba_test > t1_dev).astype(int)
coste1_test = coste_medio(y_test, pred1_test, C_FP_1, C_FN_1)
```

```

tp = ((pred1_test == 1) & (y_test == 1)).sum(); fp = ((pred1_test == 1) & (y_test == 0)).sum()
fn = ((pred1_test == 0) & (y_test == 1)).sum(); tn = ((pred1_test == 0) & (y_test == 0)).sum()
print(f"ESCENARIO 1 – coste medio en test: {coste1_test:.4f} (trivial: {y_test.mean():.4f})")
print(f"Denegación: {pred1_test.mean():.2%} | recall morosos: {tp/(tp+fn):.1%} | precisión denegaciones: {tp/max(tp+fp, 1):.1%}")
print(pd.DataFrame([[tn, fp], [fn, tp]],
                    index=["real 0", "real 1"], columns=["pred 0 (conceder)", "pred 1 (denegar)"]))

```

[celda 37]. Este bloque es, literalmente, el momento del notebook donde se “gasta” el conjunto de test: es la única vez que `X_test/y_test` se usan para calcular un coste que se va a reportar como resultado final. Tres detalles técnicos importan aquí:

1. **`modelo_dev = crea_lgbm().fit(X_dev, y_dev)`** reentrena un modelo *nuevo* sobre el 100% de `X_dev` (no reutiliza ninguno de los cinco modelos parciales que produjo la validación cruzada de la celda 34 — esos modelos, entrenados sobre 4/5 de los datos cada uno, ya cumplieron su función de generar predicciones OOF y se descartan). Este modelo, `modelo_dev`, es el que se reutiliza en las secciones 5 a 9 para toda la auditoría, así que “modelo ganador” en este notebook significa siempre este objeto concreto.
2. **`t1_dev` se usa tal cual, sin recalcularlo sobre test.** Es la aplicación práctica del esquema de validación de la sección 3.3: el umbral se congeló usando solo información de desarrollo (concretamente, de las predicciones OOF), y aquí simplemente se aplica.
3. La matriz de confusión se construye manualmente con una tabla 2x2 de `pandas.DataFrame` en vez de usar `sklearn.metrics.confusion_matrix`, para que las filas y columnas queden rotuladas exactamente con la semántica de negocio (“pred 1 (denegar)”) en vez de con los índices numéricos genéricos que produce la función de `sklearn`.

Parte 5 — Modelado, escenario 2 (C_FP = 1, C_FN = 10)

5.1 La comparativa de siete estrategias cost-sensitive

```
C_FP_2, C_FN_2 = 1, 10
T_TEORICO_2 = C_FP_2 / (C_FP_2 + C_FN_2) # 1/11 ≈ 0,0909

# OOF de las variantes cost-sensitive (mismos folds que en la sección 4)
oof["LightGBM compacto spw=10"] = cross_val_predict(
    crea_lgbm(scale_pos_weight=10), X_dev, y_dev, cv=cv, method="predict_proba")[:, 1]
oof["Logística cw={0:1,1:10}"] = cross_val_predict(
    crea_pipe_logistica(class_weight={0: 1, 1: 10}), X_dev, y_dev, cv=cv, method="predict_proba")[:, 1]

def evalua_estrategia(nombre_modelo, umbral, etiqueta):
    proba = oof[nombre_modelo]
    pred = (proba > umbral).astype(int)
    return {"estrategia": etiqueta, "umbral": umbral,
            "coste_OOF_E2": coste_medio(y_dev, pred, C_FP_2, C_FN_2),
            "denegación": pred.mean(),
            "Brier": brier_score_loss(y_dev, proba)}

t2_dev, _ = umbral_optimo(y_dev, oof_ganador, C_FP_2, C_FN_2)
t2_spw, _ = umbral_optimo(y_dev, oof["LightGBM compacto spw=10"], C_FP_2, C_FN_2)
t2_log, _ = umbral_optimo(y_dev, oof["Regresión logística L2"], C_FP_2, C_FN_2)
t2_lcw, _ = umbral_optimo(y_dev, oof["Logística cw={0:1,1:10}"], C_FP_2, C_FN_2)

tabla_e2 = pd.DataFrame([
    evalua_estrategia(NOMBRE_GANADOR, 0.5, "A. LGBM neutro + umbral 0,5 (naive)",
    evalua_estrategia(NOMBRE_GANADOR, T_TEORICO_2, "B. LGBM neutro + umbral teórico 1/11"),
    evalua_estrategia(NOMBRE_GANADOR, t2_dev, "C. LGBM neutro + umbral empírico OOF"),
    evalua_estrategia("LightGBM compacto spw=10", 0.5, "D. LGBM spw=10 + umbral 0,5"),
    evalua_estrategia("LightGBM compacto spw=10", t2_spw, "E. LGBM spw=10 + umbral empírico OOF"),
    evalua_estrategia("Regresión logística L2", t2_log, "F. Logística neutra + umbral empírico"),
    evalua_estrategia("Logística cw={0:1,1:10}", t2_lcw, "G. Logística cw10 + umbral empírico"),
]).set_index("estrategia").round(4)
tabla_e2
```

[celda 40]. Este es el bloque que responde directamente a un requisito explícito del enunciado del proyecto: comparar el ajuste de umbral en post-procesado contra el entrenamiento cost-sensitive (scale_pos_weight/class_weight), documentando el trade-off y no solo el resultado final. El diseño de la comparación merece explicarse en detalle:

scale_pos_weight=10 en LightGBM y class_weight={0: 1, 1: 10} en la logística son, pese a su sintaxis distinta, la misma idea matemática: multiplicar por 10 el peso de cada observación positiva en la función de pérdida durante el entrenamiento, de modo que el modelo “vea” un dataset diez veces más balanceado del que realmente es. Se calculan sus predicciones OOF con `cross_val_predict` reutilizando el mismo objeto `cv` de la celda 34 (los mismos folds), y se almacenan en el mismo diccionario `oof` que ya contenía las cinco variantes neutras, para poder comparar todo desde una única estructura de datos.

evalua_estrategia como función auxiliar de una sola responsabilidad: dado un nombre de modelo y un umbral, aplica la decisión y calcula coste, tasa de denegación y Brier score. Esto permite construir la tabla `tabla_e2` como una lista de siete llamadas casi declarativas, cada una etiquetada con una letra (A a G) que después se referencia en la interpretación en prosa — un patrón que hace el código autoexplicativo sin necesidad de comentarios adicionales en cada línea.

Las siete filas cubren exactamente el espacio de comparación que pide el enunciado: - **A** es el error clásico de no ajustar el umbral en absoluto (usar 0,5 con una matriz asimétrica). - **B** aplica el umbral teórico 1/11 derivado analíticamente en la sección 1, sin ajustarlo empíricamente — sirve para medir cuánto se pierde por no optimizar el umbral con datos, solo con teoría. - **C** es la estrategia elegida (umbral empírico sobre el modelo neutro). - **D** y **E** son el análogo cost-sensitive: mismo modelo reentrenado con pesos, primero con umbral naive (D) y después con su propio umbral óptimo (E) — el par D/E replica el mismo contraste A/C pero para el modelo ponderado, permitiendo ver si el ajuste de umbral sigue siendo necesario incluso cuando ya se ha reponderado el entrenamiento (la respuesta, según los resultados, es que sí: D es mucho peor que E). - **F** y **G** repiten el mismo patrón para la regresión logística, dando una tercera familia de comparación.

5.2 Diagrama de fiabilidad (calibración)

```
from sklearn.calibration import calibration_curve

fig, ax = plt.subplots(figsize=(6.5, 4.5))
for nombre, estilo in [(NOMBRE_GANADOR, "o-"), ("LightGBM compacto spw=10", "s--")]:
    frac_pos, media_pred = calibration_curve(y_dev, oof[nombre], n_bins=15, strategy="quantile")
    ax.plot(media_pred, frac_pos, estilo, ms=4, label=nombre)
ax.plot([0, 1], [0, 1], color="gray", lw=1, label="calibración perfecta")
ax.axvline(T_TEORICO_2, color="crimson", ls=":", lw=1, label="umbral teórico 1/11")
ax.set_xlabel("probabilidad predicha (OOF)")
ax.set_ylabel("fracción real de morosos")
ax.set_title("Diagrama de fiabilidad: el modelo neutro está calibrado; el ponderado no")
ax.legend(fontsize=8)
plt.tight_layout(); plt.show()
```

[celda 42]. `calibration_curve` con `strategy="quantile"` (en vez de "uniform") divide las predicciones OOF en 15 grupos con **igual número de observaciones** por grupo, no en 15 intervalos de igual anchura de probabilidad — una elección importante porque, con solo un 6,7% de positivos, casi todas las probabilidades predichas se concentran cerca de 0, y bandas de igual anchura dejarían la mayoría de bandas casi vacías, produciendo una curva ruidosa e ilegible. Para cada grupo se calcula la media de las probabilidades predichas (`media_pred`) y la proporción real de positivos observada en ese grupo (`frac_pos`); si el modelo está bien calibrado, ambos valores deberían coincidir y la curva debería seguir la diagonal $y=x$ (dibujada explícitamente como referencia). Superponer el modelo neutro y el ponderado con `scale_pos_weight=10` en el mismo gráfico es lo que permite ver a simple vista que el primero se ciñe a la diagonal y el segundo no — el ponderado sistemáticamente sobreestima el riesgo, porque sus probabilidades ya no son estimaciones de $P(\text{mora})$ sino una cantidad deformada por el reponderamiento de la pérdida durante el entrenamiento.

5.3 Evaluación única en test del escenario 2

```
# Evaluación única en test del escenario 2 (mismo modelo ya entrenado, umbral congelado OOF)
pred2_test = (proba_test > t2_dev).astype(int)
coste2_test = coste_medio(y_test, pred2_test, C_FP_2, C_FN_2)
```

```
tp = ((pred2_test == 1) & (y_test == 1)).sum(); fp = ((pred2_test == 1) & (y_test == 0)).sum()
fn = ((pred2_test == 0) & (y_test == 1)).sum(); tn = ((pred2_test == 0) & (y_test == 0)).sum()
print(f"ESCENARIO 2 – umbral OOF: {t2_dev:.4f} | coste medio en test: {coste2_test:.4f} (trivial: {10*y_test.mean():.4f})")
print(f"Denegación: {pred2_test.mean():.2%} | recall morosos: {tp/(tp+fn):.1%} | precisión denegaciones: {tp/max(tp+fp, 1):.1%}")
print(pd.DataFrame([[tn, fp], [fn, tp]],
                    index=["real 0", "real 1"], columns=["pred 0 (conceder)", "pred 1 (denegar)"])))
```

[celda 43]. La decisión de diseño más importante de esta celda es implícita y fácil de pasar por alto: **proba_test es la misma variable calculada en la celda 37**, del mismo `modelo_dev`. No se reentrena ningún

modelo nuevo para el escenario 2 — solo cambia el umbral ($t2_dev$ en vez de $t1_dev$). Esta reutilización es la que sustenta la afirmación central del notebook, repetida en varias secciones, de que “un solo modelo, dos políticas” resuelve ambos escenarios: el código lo demuestra literalmente al no llamar a `.fit()` una segunda vez.

Parte 6 — Comparación de escenarios y sensibilidad a la matriz de coste

6.1 Tabla resumen y curvas de coste lado a lado

```
resumen_esc = pd.DataFrame({
    "Escenario 1 (1:1)": {
        "umbral": round(t1_dev, 4), "denegación_test": f"{pred1_test.mean():.2%}",
        "recall_morosos": f"{((pred1_test==1)&(y_test==1)).sum()/max((y_test==1).sum(),1):.1%}",
        "FP_test": int(((pred1_test==1)&(y_test==0)).sum()),
        "FN_test": int(((pred1_test==0)&(y_test==1)).sum()),
        "coste_test": round(coste1_test, 4), "coste_trivial": round(float(y_test.mean()), 4)},
    "Escenario 2 (1:10)": {
        "umbral": round(t2_dev, 4), "denegación_test": f"{pred2_test.mean():.2%}",
        "recall_morosos": f"{((pred2_test==1)&(y_test==1)).sum()/max((y_test==1).sum(),1):.1%}",
        "FP_test": int(((pred2_test==1)&(y_test==0)).sum()),
        "FN_test": int(((pred2_test==0)&(y_test==1)).sum()),
        "coste_test": round(coste2_test, 4), "coste_trivial": round(float(10*y_test.mean()), 4)},
})
resumen_esc
```

[celda 46]. Esta tabla no recalcula nada: reempaqueta variables ya calculadas en las secciones 4 y 5 en un único DataFrame de dos columnas, una por escenario. Su valor no es computacional sino de comunicación — es la primera vez en el notebook que ambos escenarios aparecen uno junto al otro, lo que permite al lector comparar de un vistazo cómo la misma variable (umbral, denegación, FP, FN) cambia radicalmente entre columnas pese a que el modelo subyacente es idéntico.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, (cfp, cfn, t_opt, titulo) in zip(axes, [(1, 1, t1_dev, "Escenario 1 (1:1)"),
                                              (1, 10, t2_dev, "Escenario 2 (1:10)")):
    tg, cg = curva_coste(y_dev, oof_ganador, cfp, cfn)
    ax.plot(tg[1:], cg[1:], lw=1)
    ax.axvline(t_opt, color="crimson", ls="--", label=f"óptimo = {t_opt:.3f}")
    ax.axvline(cfp/(cfp+cfn), color="gray", ls=":", label=f"teórico = {cfp/(cfp+cfn):.3f}")
    ax.set_xlabel("umbral"); ax.set_ylabel("coste medio 00F"); ax.set_title(titulo); ax.legend()
axes[1].set_xlim(-0.02, 0.6)
plt.tight_layout(); plt.show()
```

[celda 47]. Dibujar las dos curvas de coste (reutilizando `curva_coste`, ya explicada en 4.1) en subplots contiguos con la misma variable en el eje X (el umbral) permite comparar directamente su **forma**: la del escenario 1 es ancha y casi plana cerca del mínimo (explicando por qué el óptimo empírico, 0,58, difiere bastante del teórico, 0,5, sin apenas penalización de coste), mientras que la del escenario 2 es mucho más estrecha y puntiaguda. `axes[1].set_xlim(-0.02, 0.6)` recorta el eje X del segundo panel porque, con el óptimo situado en torno a 0,09, el resto del rango entre 0,6 y 1 no aporta información visual relevante y solo comprimiría la parte interesante de la curva.

6.2 Análisis de sensibilidad al ratio de coste

```
# Sensibilidad: ¿cómo cambia la política óptima si el comité de riesgos cambia C_FN?
ratios = np.arange(1, 21)
sens = []
for r in ratios:
    t_r, c_r = umbral_optimo(y_dev, oof_ganador, 1, r)
```

```

sens.append({"C_FN": r, "umbral_optimo": t_r, "umbral_teorico": 1/(1+r),
            "denegación": (oof_ganador > t_r).mean(), "coste_OOF": c_r})
sens = pd.DataFrame(sens)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(sens["C_FN"], sens["umbral_optimo"], "o-", ms=4, label="óptimo empírico (OOF)")
axes[0].plot(sens["C_FN"], sens["umbral_teorico"], "--", color="gray", label="teórico 1/(1+C_FN)")
axes[0].set_xlabel("C_FN (con C_FP = 1)"); axes[0].set_ylabel("umbral"); axes[0].legend()
axes[0].set_title("El umbral óptimo sigue a la teoría")
axes[1].plot(sens["C_FN"], 100*sens["denegación"], "o-", ms=4, color="#C44E52")
axes[1].set_xlabel("C_FN (con C_FP = 1)"); axes[1].set_ylabel("% de solicitudes denegadas")
axes[1].set_title("La tasa de denegación que implica cada matriz de coste")
plt.tight_layout(); plt.show()
sens.round(4)

```

[celda 48]. Este análisis va más allá de lo que pide estrictamente el enunciado (que solo especifica dos escenarios, $C_{FN} = 1$ y $C_{FN} = 10$) para responder a una pregunta de negocio más general y más útil en la práctica: ¿qué pasaría si el comité de riesgos decidiera mañana que el ratio correcto es 7 en vez de 10? Barrer C_{FN} de 1 a 20 con `np.arange(1, 21)` y, para cada valor, llamar de nuevo a `umbral_optimo` sobre las **mismas predicciones OOF ya calculadas** (`oof_ganador`, sin volver a entrenar nada) es posible precisamente porque `umbral_optimo` separa el coste de las probabilidades: cambiar la matriz de coste es una operación de post-procesado sobre probabilidades ya fijadas, no algo que requiera reentrenar el modelo — la misma idea que sostiene la comparación C vs. E de la sección 5.1. El resultado de este barrido (que el umbral empírico sigue de cerca al teórico en todo el rango, y que la sensibilidad de la tasa de denegación es fuerte entre $C_{FN}=1$ y $C_{FN}\approx 8$ y después se satura) es lo que permite argumentar, en la interpretación de la sección, que el valor concreto “10” del enunciado no es una elección mágica y que el sistema es robusto a renegociaciones razonables de esa cifra.

Parte 7 — Auditoría (I): modelo subrogado con reglas

7.1 El extractor de reglas: recorrido recursivo del árbol interno de sklearn

```
# --- Helper: extracción de reglas legibles de un árbol sklearn ---
from sklearn.tree import DecisionTreeClassifier, plot_tree, _tree

def reglas_desde_arbol(arbol, nombres, X_ref, decisiones_ref):
    """Convierte cada hoja del árbol en una regla 'SI ... ENTONCES ...' y la anota con
    su cobertura y su acuerdo con la caja negra sobre un conjunto de referencia."""
    t = arbol.tree_
    reglas = []
    def recorre(nodo, condiciones):
        if t.feature[nodo] == _tree.TREE_UNDEFINED:
            reglas.append((condiciones, int(np.argmax(t.value[nodo][0])), nodo))
            return
        f, u = nombres[t.feature[nodo]], t.threshold[nodo]
        recorre(t.children_left[nodo], condiciones + [f"{f} ≤ {u:.3g}"])
        recorre(t.children_right[nodo], condiciones + [f"{f} > {u:.3g}"])
    recorre(0, [])
    hoja_de = arbol.apply(X_ref)
    filas = []
    for condiciones, clase, nodo in reglas:
        m = hoja_de == nodo
        filas.append({"regla": " Y ".join(condiciones),
                      "decisión": "DENEGAR" if clase == 1 else "conceder",
                      "cobertura": m.mean(),
                      "acuerdo_caja_negra": (np.asarray(decisiones_ref)[m] == clase).mean() if m.any() else np.nan})
    return pd.DataFrame(filas).sort_values("cobertura", ascending=False).reset_index(drop=True)

print("Helper de reglas listo")
```

[celda 51]. Esta función accede directamente a la representación interna de un árbol entrenado por sklearn (`arbol.tree_`) en vez de usar únicamente utilidades de alto nivel como `export_text`, porque necesita **anotar cada regla con su cobertura y su acuerdo con la caja negra sobre un conjunto de referencia distinto del de entrenamiento** — algo que ninguna utilidad estándar de sklearn calcula por sí sola.

`arbol.tree_` expone arrays paralelos de bajo nivel: `t.feature[nodo]` (el índice de la variable usada para partir en ese nodo, o el centinela `_tree.TREE_UNDEFINED` si el nodo es una hoja), `t.threshold[nodo]` (el punto de corte), `t.children_left[nodo]` y `t.children_right[nodo]` (los índices de los nodos hijos) y `t.value[nodo]` (el recuento de observaciones de entrenamiento de cada clase que cayeron en ese nodo). La función recorre es una búsqueda en profundidad clásica (recursiva) sobre esta estructura: en cada nodo interno, añade una condición legible en lenguaje natural (`f"{f} ≤ {u:.3g}"` o su complementaria `>`) a la lista de condiciones acumuladas y desciende por ambas ramas; al llegar a una hoja (`t.feature[nodo] == TREE_UNDEFINED`), registra la conjunción completa de condiciones que lleva hasta ella junto con la clase mayoritaria de esa hoja (`np.argmax(t.value[nodo][0])`).

Una vez enumeradas todas las reglas (una por hoja), `arbol.apply(X_ref)` devuelve, para cada fila de un conjunto de referencia arbitrario (`X_ref`, en la práctica el conjunto de test), el índice del nodo hoja en el que termina cayendo esa fila — sin necesidad de re-evaluar manualmente las condiciones. Comparar esa asignación de hoja (`hoja_de == nodo`) con las decisiones de la caja negra sobre las mismas filas (`decisiones_ref`) es lo que permite calcular, por regla, tanto su **cobertura** (qué fracción del conjunto de referencia cae en esa hoja) como su **fidelidad local** (de las filas que caen ahí, qué fracción coincide con lo que decidió realmente el modelo caja negra) — la pieza de información más importante de toda la auditoría del subrogado,

porque una regla puede tener alta cobertura y baja fidelidad (indicando que esa región del espacio es mal aproximada por el árbol simple) o baja cobertura y alta fidelidad (una regla de nicho pero fiable).

7.2 Búsqueda de la profundidad óptima del subrogado

```
# Decisiones de la caja negra que el subrogado debe imitar
proba_dev_fit = modelo_dev.predict_proba(X_dev)[:, 1]
dec_dev_e2 = (proba_dev_fit > t2_dev).astype(int)
dec_test_e2 = (proba_test > t2_dev).astype(int)

fidelidades = []
for prof in [1, 2, 3, 4, 5, 6]:
    arbol = DecisionTreeClassifier(max_depth=prof, min_samples_leaf=200, random_state=SEED)
    arbol.fit(X_dev, dec_dev_e2)
    fidelidades.append({"profundidad": prof,
                        "fidelidad_train": (arbol.predict(X_dev) == dec_dev_e2).mean(),
                        "fidelidad_test": (arbol.predict(X_test) == dec_test_e2).mean()})
fidelidades = pd.DataFrame(fidelidades).set_index("profundidad").round(4)
```

[celda 52]. La decisión metodológica más importante de esta celda está en la primera línea de código real: **el subrogado no se entrena para predecir y_{dev} (el target real), sino para predecir dec_dev_e2 (la decisión de la caja negra)**. Esta distinción es la definición misma de lo que es un modelo subrogado global: no compete con el modelo caja negra por acertar la realidad, sino que trata de **imitar su comportamiento**, para que un humano pueda razonar sobre “qué hace el sistema” con reglas simples, aunque esas reglas no sean el modelo predictivo óptimo para el problema real.

El bucle sobre [1, 2, 3, 4, 5, 6] entrena un árbol de cada profundidad y mide su `fidelidad_train` (acuerdo con la caja negra sobre los mismos datos de entrenamiento del subrogado) y su `fidelidad_test` (acuerdo sobre un conjunto de test que ni el modelo caja negra ni el subrogado han usado jamás para ajustar sus parámetros). Reportar ambas cifras juntas, en vez de solo una, es lo que permite detectar sobreajuste del propio subrogado: si `fidelidad_train` creciera mucho más rápido que `fidelidad_test` a medida que aumenta la profundidad, eso indicaría que el árbol se está memorizando peculiaridades del conjunto de desarrollo en vez de capturar el patrón general de decisión. `min_samples_leaf=200` limita el tamaño mínimo de cada hoja precisamente para evitar ese sobreajuste incluso en las profundidades mayores del barrido.

7.3 El árbol final y sus reglas

```
# Subrogado elegido para el escenario 2: profundidad 3 (equilibrio legibilidad/fidelidad)
subrogado_e2 = DecisionTreeClassifier(max_depth=3, min_samples_leaf=200, random_state=SEED)
subrogado_e2.fit(X_dev, dec_dev_e2)

fig, ax = plt.subplots(figsize=(16, 6))
plot_tree(subrogado_e2, feature_names=list(X_dev.columns), filled=True, rounded=True,
          class_names=["conceder", "DENEGAR"], impurity=False, fontsize=8, ax=ax)
plt.title("Subrogado del escenario 2 (imita la decisión del LightGBM con umbral 0,087)")
plt.tight_layout(); plt.show()

reglas_e2 = reglas_desde_arbol(subrogado_e2, list(X_dev.columns), X_test, dec_test_e2)
pd.options.display.max_colwidth = 130
reglas_e2.round(3)
```

[celda 53]. La elección de `max_depth=3` (frente a las seis profundidades exploradas en la celda anterior) es un punto concreto en el trade-off legibilidad/fidelidad: profundidad 3 produce como máximo 8 hojas

—fácilmente inspeccionables por un humano en una sola pantalla— con una fidelidad de test del 91,4%, mientras que subir a profundidad 5 solo añade ~1,3 puntos porcentuales de fidelidad a cambio de hasta 32 hojas, ya demasiadas para servir de explicación manejable a un comité o un auditor. `impurity=False` en `plot_tree` oculta el valor de Gini de cada nodo (información técnica de `sklearn` irrelevante para el propósito de auditoría) y `class_names=["conceder", "DENEGAR"]` sustituye las etiquetas numéricas 0/1 por su significado de negocio directamente en el gráfico del árbol.

7.4 La trampa de la fidelidad con clases desbalanceadas

```
# Mismo ejercicio para el escenario 1 – y la trampa de la fidelidad con clases desbalanceadas
dec_dev_e1 = (proba_dev_fit > t1_dev).astype(int)
dec_test_e1 = (proba_test > t1_dev).astype(int)
subrogado_e1 = DecisionTreeClassifier(max_depth=3, min_samples_leaf=200, random_state=SEED)
subrogado_e1.fit(X_dev, dec_dev_e1)
```

```
fid_e1 = (subrogado_e1.predict(X_test) == dec_test_e1).mean()
fid_trivial = 1 - dec_test_e1.mean()
denegados = dec_test_e1 == 1
fid_en_denegados = (subrogado_e1.predict(X_test)[denegados] == 1).mean()
print(f"Escenario 1 – fidelidad global del subrogado: {fid_e1:.4f}")
print(f"Pero el subrogado trivial 'conceder a todos' ya tendría: {fid_trivial:.4f}")
print(f"Fidelidad SÓLO sobre las denegaciones de la caja negra: {fid_en_denegados:.1%}")
reglas_e1 = reglas_desde_arbol(subrogado_e1, list(X_dev.columns), X_test, dec_test_e1)
reglas_e1.round(3)
```

[celda 54]. Esta celda es, en cierto sentido, una **auto-auditoría de la propia metodología de fidelidad**. Repite el mismo procedimiento de la celda 53 pero para el escenario 1, donde la caja negra deniega solo al 1,5% de los clientes. La línea `fid_trivial = 1 - dec_test_e1.mean()` calcula qué fidelidad tendría un “subrogado” que ni siquiera mirara los datos y simplemente predijera “conceder” siempre — y esa fidelidad trivial resulta ser 98,5%, casi indistinguible del 99,0% que efectivamente logra el árbol de profundidad 3. La comprobación crítica está en las tres últimas líneas del bloque de cálculo: `denegados = dec_test_e1 == 1` aísla únicamente las filas donde la caja negra sí denegó, y `fid_en_denegados` mide el acuerdo del subrogado **solo sobre esas filas** — la cifra que realmente importa cuando lo que se audita es “a quién deniega el sistema”, que resulta ser mucho más baja (47,5%). Esta celda es un ejemplo deliberado de cómo una métrica de fidelidad agregada puede ser profundamente engañosa con clases desbalanceadas, y de por qué cualquier evaluación de fidelidad debería desagregarse por la clase de interés, no reportarse solo de forma global.

Parte 8 — Auditoría (II): contrafactuales

8.1 Selección determinista de los seis clientes

```
# Selección determinista de los 6 clientes
validos = X_test.dropna()
validos = validos[(validos["flag_mora_extrema"] == 0) &
                  (validos["RevolvingUtilizationOfUnsecuredLines"] <= 1.3)]
p_val = pd.Series(modelo_dev.predict_proba(validos)[:, 1], index=validos.index)
y_val = y_test.loc[validos.index]

def elige(mask, criterio):
    cand = p_val[mask]
    if criterio == "max": return cand.idxmax()
    if criterio == "min": return cand.idxmin()
    return (cand - t2_dev).abs().idxmin()          # el más cercano al umbral

clientes = {
    "C1 (real 1, denegado claro)": elige((y_val == 1) & (p_val > 0.5), "max"),
    "C2 (real 1, denegado límite)": elige((y_val == 1) & (p_val > t2_dev) & (p_val < 0.25), "cerca"),
    "C3 (real 1, FN: aprobado)": elige((y_val == 1) & (p_val < t2_dev), "max"),
    "C4 (real 0, FP: denegado)": elige((y_val == 0) & (p_val > 0.4), "max"),
    "C5 (real 0, denegado límite)": elige((y_val == 0) & (p_val > t2_dev) & (p_val < 0.25), "cerca"),
    "C6 (real 0, aprobado claro)": elige((y_val == 0), "min"),
}

perfil = validos.loc[list(clientes.values())].drop(columns=[c for c in validos.columns if c.startswith("flag_")])
perfil.insert(0, "cliente", list(clientes.keys()))
perfil["p_mora"] = p_val[list(clientes.values())].round(4).values
perfil["decisión_E2"] = np.where(perfil["p_mora"] > t2_dev, "DENEGAR", "conceder")
perfil["clase_real"] = y_val[list(clientes.values())].values
perfil.set_index("cliente")
```

[celda 57]. Antes de elegir un solo cliente, la celda restringe el universo de candidatos con `validos`: filas de test sin ningún valor ausente (`dropna()`), sin el flag de mora extrema activado, y con utilización de crédito por debajo de 1,3. Este filtro no es casual — es la aplicación práctica de la advertencia de la sección 2.5 (los valores de utilización por encima de 10 son probablemente errores de captura) y de la sección 2.4 (los códigos 96/98 no son recuentos reales): generar contrafactuales o explicaciones detalladas sobre datos que ya se sabe que son ruido de captura produciría explicaciones sin sentido de negocio, así que esos clientes quedan excluidos del universo de selección desde el principio.

La función `elige(mask, criterio)` encapsula tres formas de seleccionar un índice dentro de un subconjunto (`mask`) de clientes: "max" (el de mayor probabilidad de mora, para casos "clarísimos"), "min" (el de menor probabilidad) y "cerca" (el más próximo al umbral de decisión `t2_dev`, calculado como `(cand - t2_dev).abs().idxmin()`, para capturar casos límite donde la decisión pudo depender de un margen mínimo). Aplicar esta función seis veces con distintas combinaciones de máscara y criterio es lo que garantiza que la selección de clientes **no sea arbitraria ni fruto de mirar manualmente el dataset hasta encontrar un ejemplo bonito**, sino el resultado reproducible de una regla de selección explícita: dos casos claros de cada clase real, dos casos límite y los dos tipos de error posibles (falso positivo y falso negativo). Esta selección es enteramente determinista (no usa ningún generador aleatorio) — ejecutar la celda dos veces produce siempre los mismos seis clientes, lo que es importante porque el resto de la sección 8 y toda la sección 9 dependen de ellos.

8.2 El buscador exhaustivo de contrafactuales propio

```
# --- Helper: búsqueda exhaustiva de contrafactuales (determinista, consciente del umbral) ---
import itertools

REJILLAS = {
    "RevolvingUtilizationOfUnsecuredLines": [0.0, 0.05, 0.1, 0.2, 0.3, 0.4, 0.6, 0.8],
    "DebtRatio": [0.0, 0.1, 0.2, 0.35, 0.5, 0.75, 1.0],
    "MonthlyIncome": [1500, 3000, 5400, 8000, 12000, 20000],
    "NumberOfOpenCreditLinesAndLoans": [0, 2, 5, 8, 12, 20],
    "NumberRealEstateLoansOrLines": [0, 1, 2, 4],
    "NumberOfTime30-59DaysPastDueNotWorse": [0, 1, 2],
    "NumberOfTime60-89DaysPastDueNotWorse": [0, 1, 2],
    "NumberOfTimes90DaysLate": [0, 1, 2],
}

ESCALA = {c: max(X_dev[c].dropna().std(), 1e-6) for c in REJILLAS}

def busca_contrafactuales(x0, modelo, umbral, k_max=4, top=3, sentido="bajar"):
    """Enumera cambios de 1..k_max variables accionables sobre rejillas plausibles.
    Devuelve los contrafactuales válidos (p<umbral si sentido='bajar'; p>umbral si 'subir')
    ordenados por (nº de cambios, distancia normalizada). Sin aleatoriedad: reproducible."""
    candidatos = []
    for k in range(1, k_max + 1):
        for combo in itertools.combinations(REJILLAS, k):
            for valores in itertools.product(*[REJILLAS[c] for c in combo]):
                if all(np.isclose(x0[c], v) for c, v in zip(combo, valores)):
                    continue
                candidatos.append(dict(zip(combo, valores)))
    Xc = pd.DataFrame([x0.to_dict()] * len(candidatos))
    for i, cambio in enumerate(candidatos):
        for c, v in cambio.items():
            Xc.iloc[i, Xc.columns.get_loc(c)] = v
    Xc = Xc[Xc.index.tolist()]
    p = modelo.predict_proba(Xc)[:, 1]
    ok = p < umbral if sentido == "bajar" else p > umbral
    res = []
    for i in np.flatnonzero(ok):
        cambio = {c: v for c, v in candidatos[i].items() if not np.isclose(x0[c], v)}
        if not cambio:
            continue
        dist = sum(abs(x0[c] - v) / ESCALA[c] for c, v in cambio.items())
        res.append((len(cambio), dist, p[i], cambio))
    res.sort(key=lambda r: (r[0], r[1]))
    vistos, finales = set(), []
    for n, d, p_, cambio in res:
        clave = tuple(sorted(cambio))
        if clave in vistos:
            continue
        vistos.add(clave)
        finales.append({"n_cambios": n, "p_contrafactual": round(float(p_), 4), "cambios": cambio})
    if len(finales) == top:
        break
```

```

return finales

def muestra_cfs(nombre, idx, cfs):
    x0 = X_test.loc[idx]
    print(f"--- {nombre} (p actual = {modelo_dev.predict_proba(X_test.loc[[idx]])[:, 1][0]:.4f}) ---")
    if not cfs:
        print(" SIN contrafactual válido dentro de los límites de plausibilidad")
    for j, cf in enumerate(cfs):
        cambios = ", ".join(f"{c}: {x0[c]:.3g} → {v:.4g}" for c, v in cf["cambios"].items())
        print(f" CF{j+1} ({cf['n_cambios']} cambio(s), p → {cf['p_contrafactual']}): {cambios}")

print("Buscador de contrafactuales listo")

```

[celda 59]. Este es, junto con `curva_coste`, el helper algorítmicamente más elaborado del notebook, y su motivación se explica ya en el enunciado de la sección: se construyó como contraste **determinista y auditable** frente a `dice-ml`, cuyo comportamiento resultó tener una sorpresa (ver 8.4) que solo se descubrió leyendo su código fuente. Conviene entender el algoritmo en cuatro fases:

Fase 1 — definición de REJILLAS. En vez de buscar contrafactuales sobre el espacio continuo completo de cada variable (lo que exigiría algún método de optimización con gradiente o heurísticas de búsqueda local), se restringe cada variable accionable a un conjunto pequeño y explícito de valores “razonables” — por ejemplo, la utilización de crédito solo puede tomar los valores [0.0, 0.05, 0.1, 0.2, 0.3, 0.4, 0.6, 0.8]. Esta discretización tiene dos ventajas: hace la búsqueda exhaustiva computacionalmente viable, y evita generar contrafactuales con precisión espuria (del tipo “reduzca su ratio de deuda a 0,34172”), que serían tanto menos creíbles para un cliente real como más difíciles de verificar como “plausibles” por parte de un auditor.

Fase 2 — enumeración de candidatos con `itertools.combinations` + `itertools.product`. El bucle exterior `for k in range(1, k_max + 1)` recorre el número de variables a cambiar simultáneamente (de 1 a 4, por defecto). Para cada tamaño `k`, `itertools.combinations(REJILLAS, k)` enumera **qué subconjunto** de `k` variables (de las 8 disponibles) se va a modificar, y para cada subconjunto elegido, `itertools.product(*[REJILLAS[c] for c in combo])` enumera el **producto cartesiano completo** de valores posibles de esas `k` variables — es decir, todas las combinaciones de valores de la rejilla para ese subconjunto concreto. La condición `if all(np.isclose(x0[c], v) ...): continue` descarta la combinación trivial en la que ningún valor cambia realmente respecto al cliente original. El resultado es una lista de diccionarios `{variable: nuevo_valor}`, cada uno un contrafactual candidato.

Fase 3 — evaluación en bloque con `predict_proba`. En vez de invocar al modelo una vez por candidato (lo que sería extremadamente lento en Python puro), se construye de una sola vez un `DataFrame` `Xc` con tantas filas como candidatos — cada fila es una copia del cliente original con las variables del candidato correspondiente sobrescritas — y se llama a `modelo.predict_proba(Xc)` **una única vez** sobre todo el bloque. Esto explota la vectorización interna de `LightGBM`: evaluar miles de filas de una sola vez es órdenes de magnitud más rápido que miles de llamadas individuales.

Fase 4 — filtrado, ranking y deduplicación. De los candidatos evaluados, `ok = p < umbral` (`0 p > umbral` si `sentido="subir"`, usado para los contrafactuales inversos de la sección 8.5) selecciona los que efectivamente cruzan la política de decisión. Para cada uno válido, se calcula `dist`, una distancia **normalizada por la desviación típica de cada variable en desarrollo** (`ESCALA[c] = X_dev[c].dropna().std()`): sin esta normalización, una variable con escala grande (como `MonthlyIncome`, en miles) dominaría artificialmente la métrica de distancia frente a una variable con escala pequeña (como `NumberOfTimes90DaysLate`, entre 0 y 20), aunque ambas representasen cambios igual de “grandes” en términos relativos. Los candidatos se ordenan por (número de cambios, distancia) — es decir, se prioriza la explicación más simple (menos variables tocadas) y, entre las igualmente simples, la más cercana al perfil original — exactamente el criterio de “cambio mínimo” que define un buen contrafactual según la literatura de XAI (Wachter et al., 2017). La deduplicación por `clave = tuple(sorted(cambio))` evita devolver dos contrafactuales que cambian exactamente el mismo

conjunto de variables con prácticamente el mismo resultado.

Complejidad. Con $k_{\max}=4$ y 8 variables en REJILLAS, el número de combinaciones de variables a considerar es $C(8,1) + C(8,2) + C(8,3) + C(8,4) = 8 + 28 + 56 + 70 = 162$, y cada una de ellas se multiplica por el producto cartesiano de valores de la rejilla correspondiente (hasta $8^4 \approx 4096$ para combinaciones de 4 variables con 8 valores cada una) — un número grande pero perfectamente manejable en un par de segundos gracias a la evaluación vectorizada de la fase 3, como confirman los tiempos de ejecución medidos durante el desarrollo (2-3 segundos por cliente).

8.3 Aplicación de la búsqueda propia a los clientes denegados

```
# Contrafactuales (búsqueda propia) para los 4 clientes DENEGADOS bajo la política E2
resultados_cf = {}
for nombre in ["C1 (real 1, denegado claro)", "C2 (real 1, denegado límite)",
              "C4 (real 0, FP: denegado)", "C5 (real 0, denegado límite)"]:
    idx = clientes[nombre]
    resultados_cf[nombre] = busca_contrafactuales(validos.loc[idx], modelo_dev, t2_dev)
    muestra_cfs(nombre, idx, resultados_cf[nombre])
print()
```

[celda 60]. Se aplica el buscador únicamente a los cuatro clientes cuya decisión bajo la política del escenario 2 es “denegar” — tiene sentido pedir un contrafactual solo a quien fue rechazado, porque es la pregunta que un cliente real haría (“¿qué tendría que cambiar para que me lo concedieran?”); a los dos clientes aprobados (C3 y C6) se les aplica en la celda 63 la pregunta inversa. El resultado revela un caso instructivo: para C4 (el falso positivo con ingresos altos pero mal historial reciente) la función devuelve una lista vacía — no existe, dentro de las cuatro variables combinadas como máximo y los valores de la rejilla considerados, ningún cambio “plausible” que baje su probabilidad por debajo del umbral. Este resultado negativo es en sí mismo información de auditoría valiosa: significa que la denegación de C4 es estructural, no el resultado de un factor aislado fácilmente corregible.

8.4 Contraste con DiCE y el descubrimiento del reseteo silencioso del umbral

```
# Contraste con DiCE (fresco + semilla en cada llamada para reproducibilidad;
# stopping_threshold = NUESTRO umbral, no el 0,5/0,25 por defecto – ver nota de la intro)
import io, contextlib
import dice_ml

datos_dice = X_dev.dropna().copy()
datos_dice[TARGET] = y_dev.loc[datos_dice.index].values
interfaz_datos = dice_ml.Data(dataframe=datos_dice,
                              continuous_features=[c for c in X_dev.columns if not c.startswith("flag_")],
                              outcome_name=TARGET)
interfaz_modelo = dice_ml.Model(model=modelo_dev, backend="sklearn")

ACCIONABLES = list(REJILLAS) # edad, dependientes y flags quedan fijos: no son accionables
RANGOS_PLAUSIBLES = {"RevolvingUtilizationOfUnsecuredLines": [0, 1.2], "DebtRatio": [0, 2],
                    "MonthlyIncome": [400, 30000], "NumberOfOpenCreditLinesAndLoans": [0, 30],
                    "NumberRealEstateLoansOrLines": [0, 10],
                    "NumberOfTime30-59DaysPastDueNotWorse": [0, 6],
                    "NumberOfTime60-89DaysPastDueNotWorse": [0, 6],
                    "NumberOfTimes90DaysLate": [0, 6]}

dice_cf = {}
```

```

for nombre in resultados_cf:
    idx = clientes[nombre]
    explicador = dice_ml.Dice(interfaz_datos, interfaz_modelo, method="random")
    with contextlib.redirect_stderr(io.StringIO()):          # silencia la barra de progreso
        salida = explicador.generate_counterfactuals(
            validos.loc[[idx]], total_CFs=3, desired_class=0,
            stopping_threshold=t2_dev, features_to_vary=ACCIONABLES,
            permitted_range=RANGOS_PLAUSIBLES, random_seed=SEED)
    cf_df = salida.cf_examples_list[0].final_cfs_df
    dice_cf[nombre] = cf_df
    x0 = validos.loc[idx]
    p_cfs = modelo_dev.predict_proba(cf_df[X_dev.columns])[:, 1]
    assert (p_cfs < t2_dev).all(), "algún CF de DiCE no cruza el umbral de la política"
    print(f"--- DiCE, {nombre} ---")
    for j in range(len(cf_df)):
        cambios = ", ".join(f"{c}: {x0[c]:.3g} → {cf_df.iloc[j][c]:.4g}" for c in ACCIONABLES
                             if not np.isclose(x0[c], cf_df.iloc[j][c], atol=1e-4))
        print(f"  CF{j+1} (p → {p_cfs[j]:.4f}): {cambios}")
    print()

```

[celda 61]. Este bloque integra `dice-ml`, la librería académica de referencia para generación de contrafactuales, y documenta un hallazgo real hecho durante el desarrollo del proyecto que merece explicarse con detalle porque ilustra un riesgo genuino de auditoría de herramientas de terceros: al inspeccionar el código fuente de `dice_ml` (en concreto, el método `misc_init` de `explainer_base.py`), se descubrió que la librería **ajusta silenciosamente el parámetro `stopping_threshold`** cuando la clase deseada es 0: si se pasa un `stopping_threshold > 0,5`, lo sobrescribe internamente a 0,25, sin lanzar ningún aviso. En términos prácticos, esto significa que llamar a `generate_counterfactuals(..., desired_class=0, stopping_threshold=0.087)` funcionaría como se espera ($0,087 < 0,5$, así que no se dispara el reseteo), pero si alguien, razonando de forma aparentemente sensata, hubiera pasado el umbral como “probabilidad de que sea clase 1” en vez de como el umbral literal del argumento (por ejemplo, `stopping_threshold = 1 - t2_dev = 0.913`), la librería lo habría silenciosamente convertido en 0,25, generando contrafactuales que, evaluados con el umbral real de la política, seguirían denegados. El `assert (p_cfs < t2_dev).all()` que cierra el bloque de generación es la salvaguarda explícita añadida tras ese descubrimiento: verifica, para cada contrafactual devuelto, que su probabilidad predicha por el modelo real (no la de DiCE) efectivamente cruza el umbral de la política antes de aceptarlo, de modo que un futuro cambio silencioso de comportamiento de la librería haría fallar el notebook de forma ruidosa en vez de producir explicaciones incorrectas de forma silenciosa.

Otros detalles de esta celda:

- `method="random"` en `dice_ml.Dice` selecciona el algoritmo de generación por muestreo aleatorio con restricciones (frente a alternativas basadas en gradiente o en algoritmos genéticos que también ofrece la librería), elegido por ser el más simple de auditar y el más rápido para un modelo de árboles no diferenciable como `LightGBM` (los métodos basados en gradiente están pensados sobre todo para redes neuronales).
- `random_seed=SEED` en cada llamada, junto con instanciar un objeto `dice_ml.Dice` nuevo para cada cliente, es lo que garantiza reproducibilidad exacta: se comprobó explícitamente durante el desarrollo (ver el prototipo de verificación) que dos llamadas consecutivas con la misma semilla producen exactamente el mismo `DataFrame` de salida.
- `contextlib.redirect_stderr(io.StringIO())` suprime la barra de progreso que `dice-ml` imprime por defecto en `stderr` durante la búsqueda — puramente cosmético, para mantener limpia la salida del notebook, sin afectar al resultado.
- `permitted_range` impone, para cada variable accionable, un rango de valores plausibles (por ejemplo, la utilización de crédito entre 0 y 1,2) — el mecanismo nativo de DiCE para imponer la misma restricción de “plausibilidad” que en la búsqueda propia se resuelve con `REJILLAS`.

8.5 Contrafactuales inversos para los clientes aprobados

```
# Contrafactuales INVERSOS para los aprobados: ¿qué habría hecho denegarles?
for nombre in ["C3 (real 1, FN: aprobado)", "C6 (real 0, aprobado claro)"]:
    idx = clientes[nombre]
    cfs = busca_contrafactuales(X_test.loc[idx], modelo_dev, t2_dev, sentido="subir", k_max=2)
    resultados_cf[nombre] = cfs
    muestra_cfs(nombre, idx, cfs)
    print()
```

[celda 63]. Reutiliza exactamente el mismo helper de la sección 8.2, pero con `sentido="subir"` — es decir, buscando cambios que **aumenten** la probabilidad de mora por encima del umbral, en vez de reducirla. Aplicarlo a C3 (el falso negativo: un moroso real que el modelo aprobó) y a C6 (el aprobado más claro de los seis) responde a una pregunta de auditoría distinta mas complementaria a la de las secciones anteriores: no “¿por qué se deniega a alguien?” sino “¿qué tan cerca o lejos está este aprobado de haber sido denegado?” — la respuesta, especialmente para C3 (que con un solo cambio de variable ya cruza el umbral en sentido contrario), es evidencia directa de la inestabilidad de frontera que se discute en la interpretación de la sección.

8.6 Las cartas al cliente

La celda markdown [celda 62], colocada deliberadamente entre la búsqueda propia (celda 60) y los contrafactuales inversos (celda 63), no contiene código pero es, en un sentido real, el entregable más importante de toda la sección 8: traduce los números de las tablas de contrafactuales en cuatro cartas redactadas en lenguaje natural, una por cada cliente denegado, que responden literalmente a la pregunta del enunciado (“si un cliente pide explicaciones sobre por qué se le deniega el crédito, ¿qué información le damos?”). Cada carta cita las variables concretas que aparecieron en los contrafactuales (utilización de tarjeta, retrasos de pago, ingresos) y, cuando la búsqueda no encontró recorrido plausible (el caso de C4), la carta lo dice honestamente en vez de inventar una recomendación cosmética — una decisión de diseño consciente: es preferible una explicación sincera de “no hay cambio inmediato disponible” a una lista de acciones que no cambiarían el riesgo real del cliente.

Parte 9 — Auditoría (III): SHAP global y local

9.1 SHAP global: TreeExplainer y el gráfico de enjambre (*beeswarm*)

```
import shap

explicador_shap = shap.TreeExplainer(modelo_dev)
muestra_test = X_test.sample(3000, random_state=SEED)
sv = explicador_shap(muestra_test)

shap.plots.beeswarm(sv, max_display=13, show=False)
plt.title("SHAP global (3.000 casos de test)")
plt.tight_layout(); plt.show()

importancia = pd.Series(np.abs(sv.values).mean(0), index=muestra_test.columns).sort_values(ascending=False)
importancia.round(3)
```

[celda 66]. `shap.TreeExplainer` implementa el algoritmo **TreeSHAP** (Lundberg et al., 2020), que calcula los valores de Shapley de forma **exacta** para modelos basados en árboles —a diferencia del `KernelExplainer` genérico, que los aproxima por muestreo y sería computacionalmente inviable en un dataset de este tamaño—, explotando la estructura del árbol para calcular en tiempo polinómico (no exponencial) la contribución marginal media de cada variable a la predicción de cada observación, promediada sobre todos los posibles órdenes de “llegada” de las variables. `muestra_test = X_test.sample(3000, random_state=SEED)` calcula SHAP sobre una submuestra aleatoria de 3.000 filas de test (no las 20.934 completas) por razones de coste computacional del propio gráfico de enjambre, que se vuelve ilegible y lento de renderizar con decenas de miles de puntos superpuestos; `random_state=SEED` asegura que la submuestra sea reproducible. `sv.values` tiene forma `(3000, 13)` — una contribución SHAP por observación y por variable, en unidades de log-odds (la escala natural de salida de un clasificador de gradient boosting antes de aplicar la función logística). `importancia = pd.Series(np.abs(sv.values).mean(0), ...)` calcula la importancia global como la media del valor absoluto de la contribución de cada variable a través de las 3.000 observaciones — la definición estándar de “importancia SHAP global”, que resume en un único número por variable cuánto mueve, en promedio, la predicción del modelo con independencia del signo de ese efecto.

9.2 Dependence plots de las variables más influyentes

```
fig_vars = ["RevolvingUtilizationOfUnsecuredLines", "age",
            "NumberOfTime30-59DaysPastDueNotWorse", "MonthlyIncome"]
for v in fig_vars:
    shap.plots.scatter(sv[:, v], show=False)
    plt.gcf().set_size_inches(7, 3.2)
    plt.title(f"Dependence plot: {v}")
    plt.tight_layout(); plt.show()
```

[celda 68]. Mientras que el gráfico de enjambre de la celda anterior resume, para cada variable, la distribución completa de sus contribuciones SHAP en una franja horizontal, un *dependence plot* (`shap.plots.scatter`) dibuja, para una única variable, su valor real en el eje X contra su contribución SHAP en el eje Y — permitiendo ver la **forma exacta** de la relación que el modelo ha aprendido: si es monótona, si satura, si tiene un salto discontinuo, etc. Se eligen precisamente las cuatro variables con mayor importancia global de la celda 66 (`RevolvingUtilizationOfUnsecuredLines`, `age`, el primer contador de morosidad y `MonthlyIncome`), porque son las que más determinan la decisión del modelo y por tanto las más relevantes para entender su comportamiento en detalle. Es en este gráfico, para `MonthlyIncome`, donde se detecta la no-monotonía (el riesgo baja con el ingreso hasta cierto punto y luego vuelve a subir ligeramente) que explica los contrafactuales aparentemente perversos de la sección 8 (donde bajar los ingresos declarados, en algún caso, reducía la

probabilidad de mora predicha).

9.3 Explicación local de los seis clientes de la sección 8

```
# Explicación local de los 6 clientes de la sección 8
X_clientes = X_test.loc[list(clientes.values())]
sv_clientes = explicador_shap(X_clientes)

for i, nombre in enumerate(clientes):
    shap.plots.waterfall(sv_clientes[i], max_display=8, show=False)
    fig = plt.gcf(); fig.set_size_inches(7.5, 3.4)
    p_i = modelo_dev.predict_proba(X_clientes.iloc[[i]])[:, 1][0]
    plt.title(f"{nombre} - p(mora) = {p_i:.3f} (umbral E2 = {t2_dev:.3f})", fontsize=10)
    plt.tight_layout(); plt.show()
```

[celda 70]. Reutiliza el mismo diccionario `clientes` de la sección 8.1, garantizando que la explicación local SHAP se calcula exactamente sobre los mismos seis individuos que ya tienen contrafactuales y cartas redactadas — el punto de conexión que hace posible el cruce de la siguiente celda. `shap.plots.waterfall` descompone la predicción de un único cliente como una cascada aditiva que parte del valor base (la predicción media del modelo sobre el conjunto de referencia) y suma, variable a variable, la contribución de cada una hasta llegar a la predicción final — el formato de explicación local más intuitivo para un lector no técnico, porque hace explícito, en unidades comparables, cuánto “empuja hacia la mora” o “empuja hacia el buen pago” cada variable del expediente de ese cliente concreto.

9.4 El cruce cuantitativo entre contrafactuales y SHAP

```
# Cruce contrafactual ↔ SHAP: ¿coinciden en qué variables importan para cada cliente?
cruce = []
for i, nombre in enumerate(clientes):
    shap_top = pd.Series(np.abs(sv_clientes.values[i]), index=X_clientes.columns).nlargest(3)
    vars_cf = set()
    if nombre in resultados_cf:
        for cf in resultados_cf[nombre]:
            vars_cf |= set(cf["cambios"])
    if nombre in dice_cf and dice_cf[nombre] is not None:
        x0 = X_clientes.iloc[i]
        for _, fila_cf in dice_cf[nombre].iterrows():
            vars_cf |= {c for c in ACCIONABLES if not np.isclose(x0[c], fila_cf[c], atol=1e-4)}
    top3 = list(shap_top.index)
    accionables_top = [v for v in top3 if v in ACCIONABLES]
    cruce.append({"cliente": nombre,
                  "top-3 |SHAP|": ", ".join(top3),
                  "variables en los CF": ", ".join(sorted(vars_cf)) if vars_cf else "-",
                  "coinciden": ("sí" if any(v in vars_cf for v in accionables_top)
                                else ("n/a" if not vars_cf else "no"))})
pd.options.display.max_colwidth = 120
pd.DataFrame(cruce).set_index("cliente")
```

[celda 71]. Esta celda no es una simple yuxtaposición visual de dos técnicas de auditoría: es una **comprobación cruzada cuantitativa** que responde directamente a una pregunta que el enunciado plantea de forma implícita (¿coinciden contrafactual y SHAP en qué variables importan para un cliente?). Para cada uno de los seis clientes, `shap_top` extrae las tres variables con mayor |valor SHAP| absoluto (`.nlargest(3)`); `vars_cf` acumula, mediante la operación de unión de conjuntos `|=`, todas las variables que aparecieron modificadas en

cualquiera de los contrafactuales generados para ese cliente, tanto por el buscador propio (`resultados_cf`) como por DiCE (`dice_cf`) — combinar ambas fuentes en un único conjunto es deliberado: el objetivo es comprobar si el conjunto *más amplio posible* de variables accionables identificadas por cualquiera de los dos métodos de contrafactual coincide con las variables que SHAP señala como más influyentes, no favorecer a uno de los dos métodos. La columna "coinciden" distingue tres casos: "sí" (al menos una de las variables top-3 de SHAP, restringida a las que son accionables, aparece en algún contrafactual), "no" (ninguna coincide) y "n/a" (no hay contrafactuales disponibles para ese cliente, como ocurre con C4). Esta columna es la que sustenta, con evidencia tabulada y no solo impresión visual, la afirmación de la interpretación de que el acuerdo entre ambos métodos es alto en los casos claros y se degrada en los casos límite.

9.5 Análisis de equidad por tramo de edad

```
bandas_edad = pd.cut(X_test["age"], [20, 30, 45, 60, 110], labels=["21-30", "31-45", "46-60", "61+"])
```

```
def metricas_por_banda(pred):
    tabla = pd.DataFrame({"banda": bandas_edad, "y": y_test.values, "pred": np.asarray(pred, dtype=int)})
    return tabla.groupby("banda", observed=True).apply(
        lambda d: pd.Series({
            "n": len(d),
            "mora_real": d.y.mean(),
            "denegación": d.pred.mean(),
            "FPR (buenos denegados)": ((d.pred == 1) & (d.y == 0)).sum() / max((d.y == 0).sum(), 1),
            "FNR (morosos aprobados)": ((d.pred == 0) & (d.y == 1)).sum() / max((d.y == 1).sum(), 1)}),
        include_groups=False)
```

```
equidad_con_edad = metricas_por_banda(pred2_test)
print("Política escenario 2, modelo CON edad:")
equidad_con_edad.round(3)
```

[celda 73]. Esta sección extra (no exigida literalmente por el enunciado, pero añadida porque `age` resultó ser la tercera variable en importancia SHAP global de la celda 66 y es, en la Unión Europea, un atributo protegido en el acceso al crédito) mide el impacto de la política del escenario 2 desagregado por cuatro tramos de edad. La función `metricas_por_banda`, diseñada para reutilizarse (se vuelve a llamar en la celda 74 con predicciones distintas), calcula por cada banda no solo la tasa de denegación bruta sino, crucialmente, el **FPR** (tasa de falsos positivos: de los buenos pagadores de esa banda, cuántos son denegados) y el **FNR** (tasa de falsos negativos: de los morosos de esa banda, cuántos son aprobados). Estas dos métricas desagregadas, y no solo la tasa de denegación total, son las que en la literatura de *fairness* en machine learning (equalized odds, Hardt et al., 2016) permiten distinguir una disparidad *justificada* (bandas con más mora real reciben más denegaciones, lo cual es estadísticamente correcto) de una disparidad *añadida por el modelo* (el FPR —la probabilidad de que un buen pagador sea denegado— difiere entre bandas más de lo que la diferencia de riesgo real justificaría).

```
# ¿Y si quitamos la edad? Coste de la mitigación + nueva disparidad
oof_sin_edad = cross_val_predict(crea_lgbm(), X_dev.drop(columns="age"), y_dev,
                                cv=cv, method="predict_proba")[:, 1]
t2_sin_edad, c2_oof_sin_edad = umbral_optimo(y_dev, oof_sin_edad, C_FP_2, C_FN_2)
modelo_sin_edad = crea_lgbm().fit(X_dev.drop(columns="age"), y_dev)
proba_test_sin_edad = modelo_sin_edad.predict_proba(X_test.drop(columns="age"))[:, 1]
pred2_sin_edad = (proba_test_sin_edad > t2_sin_edad).astype(int)

print(f"Coste E2 en test – con edad: {coste2_test:.4f} | sin edad: "
      f"{coste_medio(y_test, pred2_sin_edad, C_FP_2, C_FN_2):.4f}")
equidad_sin_edad = metricas_por_banda(pred2_sin_edad)
```

```
print("\nPolítica escenario 2, modelo SIN edad:")
equidad_sin_edad.round(3)
```

[celda 74]. Esta celda no se limita a “medir” el problema, sino que ejecuta el experimento contrafactual correspondiente a nivel de sistema: reentrena el modelo completo excluyendo age de las variables de entrada (`X_dev.drop(columns="age")`), repite exactamente el mismo procedimiento de umbral óptimo OOF y evaluación en test que se usó para el modelo original, y compara tanto el coste resultante como las nuevas métricas de equidad por banda. Que este experimento requiera repetir todo el flujo de validación cruzada + umbral óptimo + evaluación — en vez de, por ejemplo, simplemente poner a cero los coeficientes de age en un modelo ya entrenado — es importante metodológicamente: eliminar una variable puede cambiar cómo el modelo usa las demás (sus *proxies*), así que la única forma correcta de medir el efecto real de quitarla es reentrenar desde cero, no enmascararla a posteriori.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
for ax, (tabla, titulo) in zip(axes, [(equidad_con_edad, "con edad"), (equidad_sin_edad, "sin edad")]):
    tabla[["denegación", "FPR (buenos denegados)"]].plot(kind="bar", ax=ax, rot=0,
                                                         color=["#4C72B0", "#C44E52"])
    ax.plot(range(len(tabla)), tabla["mora_real"], "ko--", ms=5, label="mora real")
    ax.set_title(f"Escenario 2, modelo {titulo}")
    ax.set_xlabel("tramo de edad"); ax.legend(fontsize=8)
plt.tight_layout(); plt.show()
```

[celda 75]. `sharey=True` en `plt.subplots` fuerza a los dos paneles (con edad / sin edad) a compartir la misma escala vertical, lo cual es imprescindible para que la comparación visual de la altura de las barras entre ambos gráficos sea válida — sin este parámetro, matplotlib autoescalaría cada panel de forma independiente y una reducción real de la disparidad podría pasar desapercibida o, peor, exagerarse visualmente por un cambio de escala no informativo. Superponer la línea de `mora_real` (la tasa de mora observada en cada banda) sobre las barras de denegación y FPR permite al lector juzgar, de un vistazo, cuánta de la disparidad observada está “justificada” por el riesgo real de cada banda y cuánta parece añadida por el modelo.

9.6 Estabilidad de las explicaciones entre escenarios

```
# ¿Cambian las explicaciones si cambiamos de escenario?
# Ruta A (elegida): mismo modelo, distinto umbral → los SHAP son IDÉNTICOS por construcción;
# lo que cambia es la frontera. Comprobamos qué parte del test cambia de decisión.
# Ruta B (descartada en §5): modelo reponderado spw=10 → ¿habrían cambiado las explicaciones?
from scipy.stats import spearmanr

cambian = (pred1_test != pred2_test).mean()
print(f"Casos de test cuya DECISIÓN cambia entre escenarios (mismo modelo): {cambian:.1%}")

modelo_spw = crea_lgbm(scale_pos_weight=10).fit(X_dev, y_dev)
sv_spw = shap.TreeExplainer(modelo_spw)(muestra_test)
imp_spw = pd.Series(np.abs(sv_spw.values).mean(0), index=muestra_test.columns)

rho = spearmanr(importancia.reindex(muestra_test.columns), imp_spw)[0]
comparacion = pd.DataFrame({"|SHAP| modelo neutro": importancia,
                           "|SHAP| modelo spw=10": imp_spw.reindex(importancia.index)}).round(3)
print(f"Correlación de Spearman entre rankings de importancia: {rho:.3f}")
comparacion
```

[celda 77]. Esta celda formaliza una distinción conceptual importante que el propio comentario del código señala explícitamente: hay **dos formas distintas** en que las explicaciones podrían “cambiar entre escenarios”, y solo se puede razonar correctamente sobre la estabilidad si se distinguen. La *ruta elegida* en este

proyecto (modelo neutro + dos umbrales distintos, secciones 4 y 5) hace que los valores SHAP sean **idénticos por definición** entre escenarios, porque proceden del mismo `modelo_dev` — lo único que cambia es la frontera de decisión sobre esas mismas probabilidades. `cambian = (pred1_test != pred2_test).mean()` cuantifica cuántas filas de test caen en la banda entre ambos umbrales (17,3%) y por tanto reciben una decisión distinta, aunque su explicación SHAP subyacente no haya cambiado en absoluto.

La segunda mitad de la celda examina la *ruta descartada* en la sección 5 (reentrenar con `scale_pos_weight=10` en vez de mover el umbral): entrena `modelo_spw` desde cero y calcula sus propios valores SHAP sobre la misma `muestra_test`, para comprobar si, **si se hubiera elegido esa alternativa**, el ranking de importancia de variables habría cambiado sustancialmente. `spearmanr` calcula la correlación de rangos de Spearman entre los dos vectores de importancia media (`importancia`, calculada en la celda 66, e `imp_spw`) — una elección deliberada frente a la correlación de Pearson porque lo que interesa comparar es el **orden relativo** de las variables (¿sigue siendo la utilización de crédito la más importante en ambos modelos?), no la magnitud exacta de sus valores SHAP, que sí difiere sistemáticamente entre un modelo neutro y uno reponderado. Un rho de 0,989 es la evidencia cuantitativa que permite afirmar, con números y no solo intuición, que incluso la alternativa descartada de reponderar el entrenamiento habría producido explicaciones cualitativamente muy parecidas — una buena noticia de gobernanza, comentada en la interpretación de la sección.

Parte 10 — Discusión: el enfoque de multi-armed bandit

```
# LinUCB contextual de dos brazos (0=conceder, 1=denegar) – escenario 2
rng_bandit = np.random.default_rng(SEED)

mediana_dev = X_dev.median()
X_imputado = X_dev.fillna(mediana_dev)
Z = ((X_imputado - X_imputado.mean()) / X_imputado.std()).to_numpy()
y_stream = y_dev.to_numpy()
orden_stream = rng_bandit.permutation(len(Z))

d_dim = Z.shape[1] + 1          # + intercepto
ALPHA = 1.0                    # intensidad de exploración
A = [np.eye(d_dim), np.eye(d_dim)]      # matrices de diseño por brazo
A_inv = [np.eye(d_dim), np.eye(d_dim)]
b = [np.zeros(d_dim), np.zeros(d_dim)]
costes_stream = np.empty(len(orden_stream))

for paso, i in enumerate(orden_stream):
    x = np.concatenate([[1.0], Z[i]])
    ucb = [A_inv[a] @ b[a] @ x + ALPHA * np.sqrt(x @ A_inv[a] @ x) for a in (0, 1)]
    accion = int(np.argmax(ucb))
    coste = 10.0 * y_stream[i] if accion == 0 else 1.0 * (1 - y_stream[i])
    A[accion] += np.outer(x, x)
    b[accion] += -coste * x          # recompensa = -coste
    A_inv[accion] = np.linalg.inv(A[accion])
    costes_stream[paso] = coste

ventana = 5000
media_movil = np.convolve(costes_stream, np.ones(ventana) / ventana, mode="valid")
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(np.arange(len(media_movil)) + ventana, media_movil, label="LinUCB (media móvil 5.000)")
ax.axhline(coste2_test, color="crimson", ls="--", label=f"supervisado + umbral (test) = {coste2_test:.3f}")
ax.axhline(10 * y_dev.mean(), color="gray", ls=":", label=f"trivial conceder a todos = {10*y_dev.mean():.3f}")
ax.set_xlabel("solicitudes procesadas"); ax.set_ylabel("coste medio (escenario 2)")
ax.set_title("El bandit paga por aprender lo que el supervisado ya sabe")
ax.legend()
plt.tight_layout(); plt.show()

print(f"Coste medio del bandit – total del stream: {costes_stream.mean():.4f}")
print(f" primeras 10.000 solicitudes: {costes_stream[:10000].mean():.4f}")
print(f" últimas 20.000 solicitudes: {costes_stream[-20000:].mean():.4f}")
```

[celda 80]. Esta celda implementa **LinUCB** (Li et al., 2010), uno de los algoritmos de bandit contextual lineal más citados, desde cero (sin depender de ninguna librería externa de bandits, que además resultaron incompatibles con el entorno Python moderno del proyecto). Merece una explicación matemática completa porque es el único algoritmo de aprendizaje online de todo el notebook, en fuerte contraste conceptual con el resto (que es enteramente *batch*).

Preparación de los datos como flujo (stream). `X_imputado = X_dev.fillna(mediana_dev)` imputa los nulos con la mediana (a diferencia de `LightGBM`, un modelo de regresión ridge lineal como el que hay detrás de `LinUCB` no maneja nulos nativamente) y `Z` estandariza cada variable (media 0, varianza 1) — la estandarización es indispensable en cualquier método basado en distancias o en productos internos como `LinUCB`, porque

de lo contrario las variables con escala mayor dominarían tanto la estimación de θ como el término de exploración. `orden_stream = rng_bandit.permutation(len(Z))` baraja el orden de llegada de las 83.734 solicitudes de desarrollo usando el generador aleatorio local `rng_bandit` (no el `rng` global), simulando que las solicitudes llegan una a una, en un orden aleatorio, como lo harían en producción real.

Estructura de datos de LinUCB: una regresión ridge bayesiana por brazo. El algoritmo mantiene, para cada uno de los dos brazos (0 = conceder, 1 = denegar), una matriz A de tamaño $(d+1) \times (d+1)$ (inicializada a la identidad, que corresponde a un prior ridge con $\lambda=1$) y un vector b de tamaño $d+1$. Estas dos estructuras son exactamente las que se usarían para resolver una regresión ridge por mínimos cuadrados de la recompensa en función del contexto: en cualquier momento, el estimador de los coeficientes de ese brazo es $\theta = A^{-1} @ b$. Mantener también A_{inv} (la inversa de A) actualizada explícitamente en cada paso, en vez de invertir A de cero cada vez, es una optimización de eficiencia estándar en implementaciones de LinUCB (aunque aquí se usa `np.linalg.inv` directamente en cada iteración, sin la actualización incremental de Sherman-Morrison que usarían implementaciones de producción a mayor escala — una simplificación razonable dado que d es pequeño, 14, y el número de iteraciones, 83.734, es perfectamente manejable).

La decisión (política UCB, *Upper Confidence Bound*). Para cada solicitud i , se construye el vector de contexto $x = [1, Z[i]]$ (con un 1 antepuesto para el intercepto), y para cada brazo a se calcula $ucb[a] = \theta_a @ x + \alpha * \sqrt{x @ A_{inv_a} @ x}$. El primer término es la recompensa esperada según el modelo lineal actual de ese brazo; el segundo es un **término de exploración**: $\sqrt{x @ A_{inv} @ x}$ es, en términos bayesianos, proporcional a la desviación típica de la estimación de la recompensa para ese contexto concreto — es grande cuando el brazo se ha probado poco en contextos parecidos a x (hay mucha incertidumbre) y pequeño cuando ya se ha acumulado mucha evidencia. $\alpha = 1.0$ controla cuánto peso se da a “explorar lo incierto” frente a “explotar lo que ya se cree rentable” — el dilema exploración-explotación central de todo problema de bandits. `accion = int(np.argmax(ucb))` elige el brazo con mayor cota superior de confianza, el criterio que da nombre al algoritmo.

El coste/recompensa y la actualización. `coste = 10.0 * y_stream[i]` if `accion == 0` else `1.0 * (1 - y_stream[i])` aplica exactamente la matriz de coste del escenario 2: si se elige “conceder” (`accion == 0`) y el cliente era realmente moroso (`y_stream[i] == 1`), el coste es 10 (un falso negativo); si se elige “denegar” (`accion == 1`) y el cliente era realmente bueno, el coste es 1 (un falso positivo). La recompensa que alimenta la actualización del bandit es `-coste (b[accion] += -coste * x)`, porque un bandit maximiza recompensa y aquí el objetivo de negocio es minimizar coste — son la misma cosa con el signo invertido. Tras observar la recompensa, se actualizan A (sumando el producto externo `np.outer(x, x)`, la actualización estándar de mínimos cuadrados recursivos) y b (sumando `recompensa * x`), y se recalcula A_{inv} para la siguiente iteración — el bandit “aprende” incrementalmente, solicitud a solicitud, y crucialmente **solo actualiza el brazo que efectivamente eligió** (`A[accion]`, `b[accion]`), nunca el otro: es la esencia del problema de bandits frente a un problema supervisado clásico — el algoritmo nunca observa qué habría pasado si hubiera elegido la acción contraria, solo el resultado de la acción que realmente tomó.

Por qué esta simulación es, si acaso, generosa con el bandit. El notebook señala en su celda markdown de interpretación ([celda 81]) una asimetría importante entre esta simulación y un despliegue real: aquí, `y_stream[i]` está disponible siempre, incluso cuando el bandit elige “denegar” — en la realidad, si se deniega un crédito, nunca se observa si ese cliente habría pagado o no (el problema de *selective labels*), así que el brazo “denegar” en un sistema real generaría muchísima menos señal de aprendizaje de la que recibe aquí. Que incluso en estas condiciones idealizadas el bandit tarde miles de solicitudes en converger y termine por encima del coste del modelo supervisado es, por tanto, una cota **optimista** a favor del bandit, no pesimista — el argumento en contra del enfoque de bandit para este problema es, si acaso, más fuerte en la práctica de lo que esta simulación por sí sola sugiere.

La media móvil como diagnóstico de convergencia. `np.convolve(costes_stream, np.ones(ventana)/ventana, mode="valid")` calcula una media móvil de ventana 5.000 sobre la serie de costes observados — la forma estándar de visualizar la curva de aprendizaje de un algoritmo online sin que el ruido solicitud a solicitud oculte la tendencia. `mode="valid"` descarta los bordes donde la ventana no cabría completa dentro de la serie, evitando artefactos de frontera en el gráfico.

Parte 11 — Generación de los ficheros de producción

11.1 Umbrales finales y modelo final

```
# 1) Umbrales finales sobre OOF de construcción completa
oof_final = cross_val_predict(crea_lgbm(), X_full, y_full, cv=cv, method="predict_proba")[:, 1]
t1_final, c1_oof_final = umbral_optimo(y_full, oof_final, C_FP_1, C_FN_1)
t2_final, c2_oof_final = umbral_optimo(y_full, oof_final, C_FP_2, C_FN_2)
print(f"Escenario 1: umbral final {t1_final:.4f} (coste OOF {c1_oof_final:.4f})")
print(f"Escenario 2: umbral final {t2_final:.4f} (coste OOF {c2_oof_final:.4f})")

# 2) Modelo final y 3) predicciones de producción
modelo_final = crea_lgbm().fit(X_full, y_full)
proba_prod = modelo_final.predict_proba(X_prod)[:, 1]
pred_prod_1 = (proba_prod > t1_final).astype(int)
pred_prod_2 = (proba_prod > t2_final).astype(int)

print(f"\nDenegaciones en producción – E1: {pred_prod_1.sum():,} ({pred_prod_1.mean():.2%}) "
      f"| E2: {pred_prod_2.sum():,} ({pred_prod_2.mean():.2%})")
print(f"Denegaciones en test (referencia) – E1: {pred1_test.mean():.2%} | E2: {pred2_test.mean():.2%}")
```

[celda 83]. Es importante notar una diferencia deliberada respecto a las secciones 4 y 5: el umbral que finalmente se aplica a producción ($t1_final$, $t2_final$) **no es** el mismo $t1_dev/t2_dev$ calculado sobre las OOF de solo x_dev (83.734 filas). Aquí se recalculan las predicciones OOF de 5 folds sobre **x_full al completo** (104.668 filas, es decir, desarrollo más test unidos) con `cross_val_predict(crea_lgbm(), X_full, y_full, cv=cv, ...)`, y el umbral óptimo se deriva de esas OOF más amplias. La razón es doblemente correcta: (a) más datos producen una estimación de umbral óptimo más estable (menos varianza), y (b) para el **modelo** que efectivamente se usa en producción (`modelo_final`, entrenado con x_full completo) tiene más sentido derivar el umbral de una validación cruzada hecha también sobre el 100% de los datos disponibles, en vez de reutilizar el umbral calculado con un modelo entrenado sobre un subconjunto menor (x_dev). El coste de test (`coste1_test`, `coste2_test`, calculados en las secciones 4 y 5 con el modelo y umbral de *desarrollo*) sigue siendo la estimación honesta que se reporta como “coste esperado en producción” —porque ese es el único número que nunca participó en la elección de ningún hiperparámetro ni umbral—, mientras que `modelo_final/t1_final/t2_final` son los que efectivamente generan las predicciones que se entregan. Esta distinción —un par modelo/umbral para *estimar* el coste honestamente, y otro par, entrenado con más datos, para *producir* las predicciones finales— es un patrón estándar y deliberado en proyectos de machine learning aplicado.

11.2 Escritura de los CSV y verificación de integridad

```
# Escritura replicando el formato del fichero original (solo cambia el target)
salida1 = df_pro.copy()
salida1[TARGET] = pred_prod_1
salida1.to_csv("cs_produccion1.csv", index=False)

salida2 = df_pro.copy()
salida2[TARGET] = pred_prod_2
salida2.to_csv("cs_produccion2.csv", index=False)

# Verificación de integridad de los entregables
for fichero, pred in [("cs_produccion1.csv", pred_prod_1), ("cs_produccion2.csv", pred_prod_2)]:
    releido = pd.read_csv(fichero)
    assert releido.shape == df_pro.shape, "dimensiones alteradas"
```

```

assert list(releido.columns) == list(df_pro.columns), "columnas alteradas"
assert releido[TARGET].isna().sum() == 0, "predicciones incompletas"
assert releido[TARGET].isin([0, 1]).all(), "valores fuera de {0,1}"
assert (releido[TARGET].values == pred).all(), "predicciones no coinciden"
assert releido.drop(columns=TARGET).equals(df_pro.drop(columns=TARGET)), "features alteradas"
print(f"{fichero}: OK - {len(releido):,} filas, {releido[TARGET].sum():,} denegaciones")

```

```
salida2.head(3)
```

[celda 84]. `salida1 = df_pro.copy()` seguido de `salida1[TARGET] = pred_prod_1` es la técnica más simple y más segura para “replicar el formato original”: se parte de una copia exacta del CSV de producción tal como se cargó al principio del notebook (celda 6) y se sobrescribe **únicamente** la columna del target, dejando el resto de columnas, su orden, sus tipos y el orden de las filas exactamente intactos — garantizando por construcción que el fichero de salida “replica el que usa el notebook de partida” (mismo id de cliente implícito por posición, mismas columnas), tal como exige el enunciado.

La segunda mitad de la celda es un bloque de verificación de integridad que **relee el fichero recién escrito desde disco** (`pd.read_csv(fichero)`) en vez de confiar en las variables ya en memoria, precisamente para detectar cualquier problema introducido en la propia escritura a CSV (por ejemplo, una codificación de caracteres distinta, un separador decimal cambiado, o una fila perdida por un salto de línea mal escapado en algún valor de texto) que un chequeo puramente en memoria no revelaría. Los cinco `assert` verifican, en orden, that las dimensiones no cambiaron, que las columnas son las mismas y están en el mismo orden, que no quedó ningún valor nulo en el target (ninguna de las 45.000 predicciones puede faltar), que todos los valores son estrictamente 0 o 1 (no, por ejemplo, `0.0/1.0` como flotantes que después pudieran comportarse de forma distinta en el sistema de corrección), que los valores leídos coinciden exactamente con las predicciones en memoria, y —el más importante desde el punto de vista de integridad— que las columnas de *features* del fichero reescrito son **idénticas, fila a fila**, a las del fichero de producción original: ninguna transformación accidental alteró los datos de entrada al escribir la salida.

Parte 12 — Reflexión final crítica

```
# Tabla resumen final (generada desde las variables, no copiada a mano)
tabla_final = pd.DataFrame({
    "Escenario 1 (FP=1, FN=1)": {
        "modelo": "LightGBM compacto (300x15)",
        "umbral producción": round(t1_final, 4),
        "coste 00F construcción": round(c1_oof_final, 4),
        "coste test (estimación producción)": round(coste1_test, 4),
        "trivial a batir": round(float(y_test.mean()), 4),
        "denegaciones producción": f"{pred_prod_1.sum():,} ({pred_prod_1.mean():.1%})",
    }
    "Escenario 2 (FP=1, FN=10)": {
        "modelo": "el mismo (otro umbral)",
        "umbral producción": round(t2_final, 4),
        "coste 00F construcción": round(c2_oof_final, 4),
        "coste test (estimación producción)": round(coste2_test, 4),
        "trivial a batir": round(float(10 * y_test.mean()), 4),
        "denegaciones producción": f"{pred_prod_2.sum():,} ({pred_prod_2.mean():.1%})",
    }
})
tabla_final
```

[celda 87]. Esta es la única celda de código de la sección 12, y su diseño encierra un principio de higiene de reporting que atraviesa todo el notebook: **ningún número que aparece en prosa se ha escrito nunca “a mano”**. La tabla se construye leyendo directamente las variables ya calculadas en las secciones 4, 5 y 11 (`t1_final`, `c1_oof_final`, `coste1_test`, `y_test.mean()`, `pred_prod_1`, y sus análogos del escenario 2) — de modo que si, en una futura re-ejecución del notebook, cualquier hiperparámetro cambiara y alterase estos valores, la tabla final se actualizaría automáticamente sin riesgo de que el resumen ejecutivo de la portada (que se completa citando estos mismos números) quedara desincronizado con lo que el código realmente produjo. El resto de la sección 12 (celdas markdown de limitaciones, riesgos de despliegue y mejoras futuras) son deliberadamente prosa sin código: es el espacio del notebook dedicado enteramente a la reflexión crítica que pide explícitamente el criterio de evaluación de “análisis y capacidad de reflexión crítica”, y no debía diluirse con más cálculos.

Cierre: reproducibilidad y cómo re-ejecutar el notebook

Todo el notebook se apoya en una única semilla global (`SEED = 42`) propagada consistentemente a cada componente estocástico: los `random_state` de `train_test_split`, `StratifiedKFold`, `LGBMClassifier`, `XGBClassifier`, `LogisticRegression`, `DecisionTreeClassifier`, `sample()` de `pandas`, y las llamadas a `dice_ml(random_seed=SEED)`; y, por separado, los generadores locales `np.random.default_rng(SEED)` para el muestreo de la sección 2 (aunque no se usa explícitamente en muchas celdas al ser las funciones de `sklearn` las que ya fijan su propio `random_state`) y para el barajado del stream del bandit en la sección 10. La combinación de estas dos prácticas —semilla única global y generadores locales encapsulados donde hace falta aleatoriedad “manual” fuera de `sklearn`— es lo que permite que dos ejecuciones completas del notebook, en cualquier máquina con las mismas versiones de librerías (fijadas en `requirements.txt`: `lightgbm>=4.3`, `xgboost>=2.0`, `shap>=0.46`, `dice-ml>=0.11`, entre otras), produzcan exactamente los mismos números, hasta la última cifra decimal, sin necesidad de fijar manualmente ninguna semilla adicional celda a celda.

El notebook se ejecuta de principio a fin con:

```
jupyter nbconvert --to notebook --execute --inplace practica_B4T2_XAI.ipynb
```

y tarda aproximadamente 5-8 minutos en un portátil moderno, dominado por las llamadas a `cross_val_predict` (varias decenas de entrenamientos de `LightGBM` y `XGBoost` repartidos entre las secciones 4, 5, 9 y 11) y por el cálculo de valores `SHAP` sobre 3.000 observaciones en la sección 9.